BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

AN ENVIRONMENT FOR THE VISUAL DESIGN, MONITORING, AND MANAGEMENT OF DISTRIBUTED CONTROL SYSTEMS AND IOT NETWORKS

PROSTŘEDÍ PRO VIZUÁLNÍ NÁVRH, MONITOROVÁNÍ A SPRÁVU DISTRIBUOVANÝCH ŘÍDICÍCH SYSTÉMŮ A IOT SÍTÍ

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **JAKUB KAPITULČÍN**
AUTOR PRÁCE

SUPERVISOR **doc. Ing. VLADIMÍR JANOUŠEK, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

Distributed IoT systems often consist of numerous different devices and communication protocols. In these systems, data flow and control logic are frequently split across multiple standalone programs and configuration files, which increases developers' mental overhead and makes it easy to overlook bugs and timing-dependent failures. This thesis presents a development environment that unifies modeling, deployment, monitoring, testing, and debugging for distributed IoT systems within a single configurable platform. The platform is divided into four main components: an Electron-based IDE for visual development and runtime inspection, an Elixir/Phoenix gateway that serves as the network entry point, an Elixir server for device management and trace storage, and a portable C++17 runtime library for executing automata across all targets. Device behavior is modeled visually as an Extended Finite State Machine with custom Lua hooks that execute scripts for guards, actions, and selected hardware interactions. Automata are stored in YAML format on the server and compiled into IR, which the runtime engine loads for execution. The implemented platform supports configurable fault injection, execution trace recording, time-travel debugging, and structural analysis of automata networks using a Petri net lift. The system was tested using unit, integration, end-to-end deployment, hardware smoke tests on embedded targets, and manual testing. The result is a customizable platform that keeps the design model, runtime execution, fault reproduction, and analysis layers connected via a single state-machine-based representation.

Abstrakt

Distribuované IoT systémy sa často skladajú z mnohých rôznych zariadení a komunikačných protokolov. V takýchto systémoch je dátový tok a riadiaca logika rozdelená medzi viacero samostatných programov a konfiguračných súborov, čo zvyšuje mentálnu záťaž vývojárov a uľahčuje prehliadnutie chýb a zlyhaní závislých od časovania. Táto práca predstavuje vývojové prostredie, ktoré zjednocuje modelovanie, nasadzovanie, monitorovanie, testovanie a ladenie distribuovaných IoT systémov do jedinej konfigurovateľnej platformy. Platforma je rozdelená na štyri hlavné komponenty: IDE postavené na Electron pre vizuálny vývoj a inšpekciu behu, bránu Elixir/Phoenix slúžiacu ako sieťový vstupný bod, Elixir server pre správu zariadení a ukladanie stôp vykonávania a prenositeľnú runtime knižnicu C++17 pre spúšťanie automatov na všetkých cieľových platformách. Správanie zariadení je modelované vizuálne ako rozšírený konečný automat s vlastnými Lua skriptmi pre stráže, akcie a vybrané hardvérové interakcie. Automaty sú následne uložené vo formáte YAML na serveroch a skompilované do IR, ktoré načítava runtime jadro pri vykonávaní. Implementovaná platforma podporuje konfigurovateľnú injekciu porúch, záznam stôp vykonávania, ladenie s návratom v čase a štrukturálnu analýzu sietí automatov pomocou zdvihu na Petriho sieť. Systém bol testovaný pomocou jednotkových, integračných a end-to-end testov, hardvérových smoke testov na vstavaných cieľoch a manuálneho testovania. Výsledkom je konfigurovateľná platforma, ktorá udržiava model návrhu, vykonávanie, reprodukciu porúch a analytickú vrstvu prepojené prostredníctvom jednotnej reprezentácie na báze stavových strojov.

Keywords

extended finite state machine, Petri net, IoT, distributed systems, embedded systems, fault injection, time-travel debugging, execution tracing, Lua

Kľúčové slová

rozšírený konečný automat, Petriho sieť, IoT, distribuované systémy, vstavané systémy, injekcia porúch, ladenie s návratom v čase, trasovanie vykonávania, Lua

Reference

KAPITULČÍN, Jakub. *An environment for the visual design, monitoring, and management of distributed control systems and IoT networks.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. Ing. Vladimír Janoušek, Ph.D.

Rozšírený abstrakt

Distribuované IoT systémy sa bežne skladajú z mnohých rôznych zariadení a komunikačných protokolov. Dátový tok a riadiaca logika sú v týchto systémoch rozdelené medzi viacero samostatných programov a konfiguračných súborov. Výsledkom je zvýšená mentálna záťaž vývojárov a ľahkosť, s ktorou možno prehliadnuť chyby a zlyhania závislé od časovania. Pri poruche vývojár zvyčajne rekonštruuje správanie systému z logov, sieťových udalostí a čiastkových pohľadov na jednotlivé zariadenia, bez možnosti systematicky zasiahnutú sekvenciu zopakovať alebo preskúmať.

Cieľom tejto práce bolo navrhnúť a implementovať vývojové prostredie, ktoré zjednocuje modelovanie, nasadzovanie, monitorovanie, testovanie a ladenie distribuovaných IoT systémov do jedinej konfigurovateľnej platformy.

Platforma pozostáva zo štyroch hlavných komponentov. IDE postavené na Electron umožňuje vizuálny vývoj a inšpekciu behu automatu. Brána Elixir/Phoenix slúži ako sieťový vstupný bod a sprostredkúva komunikáciu medzi IDE a zariadeniami cez trvalé WebSocket spojenia. Elixir server zabezpečuje správu zariadení, orchestráciu nasadení a ukladanie stôp vykonávania. Prenositeľná runtime knižnica v C++17 spúšťa automaty na všetkých cieľových platformách z jedného zdrojového stromu — od desktopového Linuxu až po ESP32 (Xtensa LX6) a NXP FRDM-MCXN947 (ARM Cortex-M33).

Správanie zariadení je modelované vizuálne ako rozšírený konečný automat. Automat má pomenované stavy, typované premenné a päť typov prechodov: strážený, časovaný, udalostný, pravdepodobnostný a okamžitý. Stráže, vstupné a výstupné akcie stavov a hodinový kód sú písané v Lua, čo umožňuje pridať výpočtovú logiku bez zmeny štruktúry automatu. Automaty sa ukladajú vo formáte YAML. Pri nasadzovaní server buď odošle YAML priamo na zariadenie, alebo ho skompiluje do prechodného formátu `aeth_ir_v1` pre zariadenia, ktoré nemajú dostatok zdrojov na parsovanie YAML.

Platforma podporuje konfigurovateľnú injekciu porúch na komunikačnej hranici. Profil poruchy umožňuje nastaviť oneskorenie, jitter, pravdepodobnosť straty správy, simulované odpojenie a vybíjanie batérie. Všetky náhodné rozhodnutia injektora porúch čerpajú zo seedovaného generátora, takže poruchové scenáre sú deterministicky reprodukovateľné. To umožňuje v laboratóriu spoľahlivo vyvolať poruchy, ktoré by sa v prevádzke objavili len zriedkavo.

Každá zmena stavu a hodnota premennej sa zaznamenáva spolu s časovými pečiatkami do stopy vykonávania. Server tieto záznamy ukladá do časových radov a voliteľne ich zrkadlí do InfluxDB. Ladenie s návratom v čase tieto záznamy využíva: operátor vyberie bod v minulosti v IDE a server rekonštruuje kompletný stav automatu pre daný okamih. Výsledok sa odošle do zariadenia ako binárna správa `RestoreState` (`0x48`), ktorá atomicky nastaví stav a premenné bez spustenia vstupných akcií. Po príkaze `Resume` zariadenie pokračuje vo vykonávaní od obnoveného stavu.

Štrukturálna analýza pomocou zdvihu na Petriho sieť dopĺňa pohľad na jednotlivé automaty o sieťovú perspektívu. Každý automat sa zdvihne na subnet Petriho siete a zdieľané kanály medzi automatmi sa stanú spojovacími oblúkmi medzi subnetmi. IDE zobrazuje výsledok ako interaktívny graf a umožňuje tri druhy štrukturálnej analýzy: detekciu mŕtvych stavov, identifikáciu úzkych miest a hlásenie súperenia o zdieľané zdroje.

Systém bol testovaný na viacerých úrovniach. Jednotkové a integračné testy pokrývajú parser automatov, runtime jadro, serverovú orchestráciu, protokol brány a vrstvu časových radov. End-to-end test nasadzuje vzorový automat cez celý stack a overuje, že výsledný stav zariadenia a emitovaná telemetria zodpovedajú očakávaniu. Hardvérové smoke testy

overujú kompiláciu a spustenie na ESP32 a MCXN947. Katalóg 39 ukážkových automatov v 15 kategóriách slúži aj ako regresný korpus.

Výsledkom je konfigurovateľná platforma, ktorá udržiava model návrhu, vykonávanie, reprodukciu porúch a analytickú vrstvu prepojené prostredníctvom jednotnej reprezentácie na báze stavových strojov. Vývojár pracuje v jednom prostredí od prvého nákresu automatu až po analýzu správania systému pri injektovaných poruchách.

An environment for the visual design, monitoring, and management of distributed control systems and IoT networks

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of doc. Ing. Vladimír Janoušek, Ph.D. I have listed all the literary sources, publications, and other sources which were used during the preparation of this thesis. I used OpenAI Codex for source-code assistance, test design, refactoring suggestions, and thesis text refinement. I used GitHub Copilot for inline code completion during implementation. I used Google Stitch for early user-interface layout exploration and visual design ideation. I used Claude (Anthropic) as an AI coding and writing assistant for implementation support, thesis text editing, and proofreading. All generated suggestions were reviewed, edited, and integrated by the author.

....
Jakub Kapitulčín
May 13, 2026

Acknowledgements

I would like to thank doc. Ing. Vladimír Janoušek, Ph.D. for his guidance throughout the work on this thesis.

Contents

List of Figures

Chapter 1

Introduction

The Internet of Things (IoT) connects an ever-increasing number of heterogeneous devices that must cooperate to execute control logic in real time. Such systems are inherently distributed and stateful: a smart thermostat cycles through heating, idle, and cooling modes; a production-line robot arm progresses through pick, move, and place phases; a sensor watchdog transitions between monitoring, alarming, and recovery states. The heterogeneity of devices, communication protocols, and data formats in large IoT deployments is a well-documented integration challenge [28]. Despite the pervasiveness of state-driven behavior, most existing IoT toolchains treat control logic as ad-hoc scripts embedded in event callbacks or workflow DAGs, making the underlying state structure implicit and difficult to reason about.

The consequence is a practical problem: when a system deployed across dozens of devices misbehaves, developers must reconstruct the global state from scattered log messages, often without any ability to replay or visually inspect the sequence of events that led to the fault. Testing distributed IoT systems reliably requires simulating faults and reproducing timing-sensitive failure scenarios [4], a capability that most existing IoT platforms do not provide natively.

This thesis addresses that problem by presenting *Aetherium Automata*, a complete toolchain for the visual design, deployment, and monitoring of distributed IoT control systems modeled as *Extended Finite State Machines* (EFSMs). The system treats automata as first-class citizens: every device behavior is expressed as a named state machine with typed variables, Lua-scripted guards and actions, and a rich set of transition types. The toolchain includes:

- a portable C++17 runtime engine that executes EFSM definitions on targets ranging from desktop Linux to ESP32 (Xtensa LX6) and the NXP FRDM-MCXN947 evaluation board [30] (ARM Cortex-M33),
- a YAML-based domain-specific language (DSL) for authoring automata,
- a real-time Elixir/Phoenix gateway that bridges the IDE and the device network,
- an Electron-based visual IDE for design, live monitoring, fault injection, and time-travel debugging,
- structural analysis using Petri net formalism for dead-state and bottleneck detection and resource contention identification across multi-automata networks.

1.1 Design Philosophy

Two overarching principles shaped every design decision in Aetherium Automata, from the choice of transition types to the structure of the Petri net analysis layer. They are stated here because understanding them makes the architectural choices in later chapters self-evident rather than arbitrary.

Reality-in-the-Loop. Distributed IoT systems exhibit failure modes that are invisible to unit tests: timing races between devices, cascading state-machine anomalies triggered by intermittent connections, and recovery paths that are never exercised under nominal conditions. Traditional testing approaches are insufficient because the bugs live in the *interactions* between components, not inside any single unit.

Reality-in-the-Loop (RitL) is the principle that testing and production share the same execution substrate. The production automaton runs on real (or emulated) hardware; a fault-injection layer perturbs the communication boundary with a configurable probability. Three features of Aetherium together close the loop:

1. **Probabilistic and stochastic transitions.** Rare real-world events — sensor dropout, power glitch, packet burst — are modeled as transitions with configurable probability weights. A transition that fires 0.1% of the time in production can be promoted to 100% for a stress run without touching the automaton definition. Events that may never occur in a real deployment can be reliably induced in the lab.

2. **Black-box contracts.** The interface under test is formally stated: input ports, output ports, observable states, and emitted events. The fault injector operates at this boundary, not inside the implementation. This means external devices and third-party subsystems can be subjected to the same testing discipline as first-party automata.

3. **Time-travel debugging.** When an injected fault reveals a misbehavior, the developer does not need to reproduce it from scratch. The execution trace is rewound to the exact state preceding the anomaly; from there, forward steps can be taken under full variable inspection. The fault is observed, not just logged.

Together, these features form a test-debug cycle that is not possible in toolchains where the state machine is implicit and the communication layer is opaque.

Deployment-Aware Development. In most IoT toolchains, developers write logic and deploy it into a communication layer they do not own. Optimization is retrospective: instrument the deployment, collect logs, correlate events across devices. The developer must infer what the network is doing.

Deployment-Aware Development (DAD) inverts this relationship. Because Aetherium owns the communication layer end-to-end — binary wire protocol, custom gateway, structured telemetry — every deployed automaton is accompanied by live metadata: round-trip time, connection type, wake-up timer intervals, and — on hardware that exposes it — battery level. This information is not scraped from logs; it appears in the same IDE panel as the running state machine. The developer can therefore make quantified, deliberate trade-offs:

- A measured RTT of 350 ms informs the choice of timer-transition intervals.

- On battery-capable targets, the protocol carries charge state; a low reading can trigger an immediate transition to a low-power mode before the device disconnects.
- A known intermittent connection can be reproduced with the fault injector to verify recovery behavior before it occurs in production.

The Petri net analysis layer is the highest expression of this principle. Lifting individual automata to a Petri net and composing them produces a single structural model of the entire network. Where the device view shows each automaton in isolation, the Petri net view reveals structural properties that emerge from their interaction: where channel places are overloaded (more producers than consumers), which state places are structural dead ends, and which automata compete for shared resources. This is analogous to acquiring a third spatial dimension: properties that are invisible when inspecting automata one at a time become immediately evident when the full network is projected onto a Petri net.

1.2 Goals

The primary goals of this thesis are:

1. Design and implement an EFSM model expressive enough to cover the control logic of real-world IoT devices, including time-driven, event-driven, and probabilistic behavior — the latter being essential for Reality-in-the-Loop testing.
2. Implement a portable C++ runtime engine capable of executing the model on both desktop and constrained embedded platforms.
3. Design a YAML domain-specific language for authoring automata definitions.
4. Implement a real-time gateway and IDE that enable an operator to deploy, monitor, and debug automata across a network of heterogeneous devices, with full deployment metadata available to developers.
5. Provide a fault injection subsystem and execution trace replay (time-travel debugging) to support Reality-in-the-Loop testing.
6. Provide a Petri net analysis layer that lifts the multi-automata network to a higher structural plane, enabling Deployment-Aware reasoning about the system as a whole.
7. Validate the system with a catalog of representative IoT showcase automata.

1.3 Thesis Structure

Section 1.1 above introduces the two guiding principles (Reality-in-the-Loop and Deployment-Aware Development) that inform all subsequent design decisions. Chapter 2 establishes the theoretical background: Finite State Machines, Extended Finite State Machines, Petri nets, and Record-and-Replay Debugging. Chapter 3 analyzes existing tools (Node-RED, Eclipse 4diac), compares programmer mental models, and derives requirements. Chapter 4 describes the overall system architecture. Chapter 5 specifies the automata model and the YAML DSL. Chapter 6 covers the C++ runtime engine, including fault injection and execution trace recording. Chapter 7 describes the gateway, binary wire protocol, and IDE integration layer. Chapter 8 describes the Petri net analysis layer. Chapter 9 presents the showcase catalog. Chapter 10 documents automated, end-to-end, and manual validation. Chapter 11 walks through an end-to-end demonstration. Chapter 12 concludes with results and future work.

Chapter 2

Background

2.1 Finite State Machines

A *Finite State Machine* (FSM) is a well-established mathematical formalism for modeling systems that can be in exactly one of a finite number of states at any given time and that change state in response to inputs [15]. Formally, a deterministic FSM is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F) \tag{2.1}$$

where:

- Q is a finite, non-empty set of states,
- Σ is a finite input alphabet (set of events or signals),
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of accepting (final) states.

In control applications, the acceptance criterion is typically not used; the focus lies instead on the transition function and any actions associated with states or transitions. Two classical variants are distinguished:

Moore machine Output depends only on the current state. Output is produced upon entering a state.

Mealy machine Output depends on both the current state and the current input. Output is produced when a transition fires.

Aetherium Automata supports both semantics simultaneously through per-state code hooks executed on entry and per-tick, and per-transition code executed when a transition fires.

2.2 Extended Finite State Machines

A plain FSM is insufficient for most practical control problems because all variation must be encoded into distinct states. An *Extended Finite State Machine* (EFSM) augments the FSM with a variable store, transforming guards and actions into functions over that store [14]:

$$M_E = (Q, \Sigma, V, \delta_E, \lambda, q_0) \tag{2.2}$$

where:

- Q, Σ, q_0 are as in the standard FSM,
- V is a finite set of typed variables (the *data context*),
- $\delta_E : Q \times \Sigma \times \mathit{Bool}(V) \to Q$ is the extended transition function; each transition edge carries a *guard* $g : V \to \mathit{Bool}$ that must be satisfied for the transition to fire,
- $\lambda : Q \times \Sigma \times V \to V$ is the output/action function that updates the variable store when a transition fires.

In Aetherium, the variable store V contains three classes of variables: *input* variables (written by the external environment, read-only to the automaton), *output* variables (written by the automaton, consumed externally), and *internal* variables (read-write by the automaton alone). Guards and actions are expressed as Lua [17, 16] scripts evaluated against V at runtime. Lua was specifically designed as a lightweight, embeddable extension language [17], making it well-suited for use inside a portable C++ runtime with constrained memory budgets.

Each state $q \in Q$ additionally carries three optional code hooks evaluated in the following order:

1. `on_enter` — executed once when the state is entered,
2. `body` — executed on every execution tick while the machine remains in the state,
3. `on_exit` — executed once immediately before leaving the state.

This structure corresponds to a Moore/Mealy hybrid: `on_enter` provides state-dependent output (Moore), while the transition `body` hook provides input-dependent output (Mealy).

2.3 Petri Nets

A *Petri net* is a mathematical formalism designed for modeling concurrent, distributed, and asynchronous systems [36]. It generalizes finite automata by allowing multiple simultaneous active conditions (tokens in multiple places). A formal survey with analysis properties is given by Murata [27]. A Petri net is defined as a 4-tuple:

$$N = (P, T, F, M_0) \tag{2.3}$$

where:

- P is a finite set of *places* (representing conditions or states),

- T is a finite set of *transitions*, with $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ is the *flow relation* (directed arcs),
- $M_0 : P \to \mathbb{N}$ is the *initial marking* (token distribution).

A transition $t \in T$ is *enabled* under marking M if every input place p of t holds at least one token: $\forall p \in \bullet t : M(p) \geq 1$. Firing t removes one token from each input place and adds one token to each output place:

$$M'(p) = M(p) - F(p,t) + F(t,p) \tag{2.4}$$

Petri nets provide the foundation for several analysis problems relevant to IoT networks:

Reachability Determine whether a given marking M' is reachable from M_0.

Boundedness Determine whether the number of tokens in any place is bounded.

Liveness (deadlock-freedom) Determine whether every transition can eventually fire from every reachable marking.

In Aetherium, the Petri net formalism is applied at the *network level*: each automaton's current state contributes a conceptual token, and the combined network of automata is analyzed for deadlocks, bottlenecks, and resource contention. This is described in detail in Chapter 8.

2.4 Record-and-Replay Debugging

Record-and-replay is a debugging strategy in which program execution is first *recorded* — capturing non-deterministic inputs, scheduling decisions, and event orderings — and then *replayed* deterministically from the recorded log to reproduce the original execution. The technique was formalized by LeBlanc and Mellor-Crummey [22], who applied it to shared-memory parallel programs by logging synchronization events; their system, *Instant Replay*, demonstrated that deterministic replay is practical even when the original execution involved data races.

Engblom surveys the broader landscape of reverse execution and debugging tools [8], distinguishing three levels of granularity:

Instruction-level Every CPU instruction is recorded and used by full-system simulators. Provides complete fidelity but imposes significant execution overhead.

System-call-level Only OS interactions are recorded; this is sufficient for many user-space bugs and is used by tools such as Mozilla `rr`, whose design paper reports low recording overhead for a range of user-space workloads on suitable Linux/x86 hardware [32].

Application-level Only domain-relevant events are recorded (messages, state transitions, variable updates). Lowest overhead; fidelity is limited to the recorded abstraction.

Geels et al. apply application-level replay to distributed systems [12]: messages between components are logged at each replica, and replay re-injects them in the same order, reproducing failures that would otherwise require weeks of manual investigation.

Aetherium adopts an application-level strategy tailored to EFSM networks. Every state transition, variable update, and inter-device message is recorded as a structured

`TraceRecord`. The IDE can step backward and forward through this trace. Beyond offline replay, Aetherium implements *live rewind*: the server reconstructs the automaton state at any past timestamp and dispatches a `RestoreState` command to the physical device, resetting it to that historical state without stopping the orchestration layer. This mechanism is described in detail in Section 6.6.

2.5 Implications for an IoT Toolchain

The three theoretical ideas above are not independent background topics; they define the shape of the toolchain. A plain FSM gives the developer explicit control over a state graph. An EFSM adds the data context needed by real devices: sensor readings, counters, retry budgets, battery state, and output commands. A Petri net then lifts multiple local automata into a structural model of the network. Record-and-replay closes the loop by turning runtime behavior back into inspectable state history.

This layering is important because each formalism answers a different engineering question. The FSM view answers „where is the device now?“ The EFSM view answers „why is it allowed to transition?“ The Petri net view answers „what happens when several devices interact?“ The replay view answers „what exact sequence produced this failure?“ Aetherium intentionally keeps all four questions tied to the same model, rather than forcing developers to switch between unrelated tools.

For example, a watchdog automaton can be understood locally as an EFSM with states `monitoring`, `alarming`, `recovering`, and `failed`. Its retry counter and silence timer belong naturally to the EFSM variable store. Once the watchdog is connected to a production-line controller, however, a local view is no longer enough: the alarm output may block a downstream actuator, consume a shared maintenance resource, or trigger a recovery command in another automaton. The Petri lift reveals these interactions as token flow through shared places.

Similarly, record-and-replay is only useful if the recorded abstraction matches the model that the developer reasons about. Capturing raw CPU instructions would be excessive for a distributed IoT IDE. Capturing only textual logs would be insufficient because logs do not reconstruct the state. Aetherium records the domain-level events that matter to the EFSM: active state, variable values, transition identifiers, message sequence numbers, and deployment metadata. This makes the replay both compact and semantically meaningful.

The resulting design choice is pragmatic: Aetherium does not attempt to become a theorem prover for arbitrary distributed systems. Instead, it provides enough formal structure to make common IoT failures visible: missing transitions, malformed state definitions, producer-consumer mismatches, unbounded queues, unavailable shared resources, intermittent communication, and timing-dependent recovery bugs. These are the problems that motivated the requirements in the next chapter.

Chapter 3

Analysis of Existing Solutions

Before designing Aetherium Automata, two representative existing platforms were analyzed: Node-RED, which represents the flow-based IoT automation category, and Eclipse 4diac, which represents the IEC 61499 function-block category. Together, they cover the main points of comparison relevant to the thesis goals: explicit state modeling, embedded portability, testability, and observability.

3.1 Node-RED

3.1.1 Overview

Node-RED [34] is a low-code, flow-based programming tool originally developed in early 2013 by Nick O'Leary and Dave Conway-Jones at IBM's Emerging Technology Services group. It was open-sourced in September 2013 and moved to the JS Foundation in 2016 [33]; after the latter foundation merger, Node-RED became part of the OpenJS Foundation ecosystem. Today, it is one of the most visible visual programming environments for IoT prototyping.

3.1.2 Programming Model

Node-RED implements the *flow-based programming* paradigm [26], in which an application is expressed as a directed graph of *nodes* connected by *wires*. Each node encapsulates a single unit of processing and communicates with adjacent nodes by passing JavaScript message objects (`msg`). Flows are the top-level grouping unit and are serialized as JSON. The browser-based editor communicates with a Node.js runtime that executes the flows.

Three kinds of built-in nodes are provided:

Input nodes Inject messages into the flow from external sources, such as timers, MQTT [31] brokers, HTTP endpoints, or hardware GPIO.

Processing nodes Transform, filter, or route messages — function nodes accept arbitrary JavaScript code, switch nodes branch on message properties.

Output nodes Forward messages to external sinks: MQTT, HTTP, or database.

The official Node-RED Library lists a large community ecosystem of contributed nodes and flows [29]. Because this catalog changes continuously, this thesis treats the exact package count as a moving operational detail rather than as a fixed property of the tool.

3.1.3 State Management

Node-RED has no built-in concept of a state machine. Application-level state must be explicitly managed using *context storage*: node context (local to a single node), flow context (shared within a flow tab), or global context (shared across all flows). Accessing and mutating context from within function nodes requires manual bookkeeping and is untyped.

Several community packages attempt to address this gap by adding FSM nodes (e.g., `node-red-contrib-finite-statemachine`), but these are third-party add-ons with no native IDE integration, no visual state graph, and no support for timed or probabilistic transitions. Representing complex stateful logic natively in Node-RED results in large, hard-to-navigate flow graphs where control and data flows are interleaved.

3.1.4 Embedded Targets and Observability

Node-RED runs on the Node.js runtime, which requires a relatively capable host: a minimum Raspberry Pi-class device with a full operating system. Bare-metal microcontrollers such as ESP32 (without an OS) or ARM Cortex-M targets are not directly supported.

Runtime observability is provided through a Debug side-panel that displays message values at selected wire taps. There is no structured execution trace, no replay capability, and no fault injection mechanism. Reproducing a timing-sensitive failure requires either manual test setup or third-party logging infrastructure.

3.1.5 Summary

Node-RED excels at rapid prototyping and data routing in environments where state is simple or absent. It is poorly suited to applications requiring rich stateful behavior, reproducible testing under adverse conditions, or deployment on constrained embedded hardware.

3.2 Eclipse 4diac

3.2.1 Overview

Eclipse 4diac [7] is an open-source IDE and runtime for IEC 61499-based distributed control. First presented by Strasser et al. [39], it is now an Eclipse Foundation project with a dedicated reference text [44].

3.2.2 IEC 61499 and Function Blocks

IEC 61499 [19] is an international standard for distributed industrial process measurement and control systems. It defines a *Function Block* (FB) as the primary computational unit and extends the earlier IEC 61131-3 [20] PLC programming standard to multi-device, distributed architectures.

Three FB types are defined:

Basic Function Block (BFB) The fundamental executable unit. Its behavior is defined by an *Execution Control Chart* (ECC) — an event-driven state machine — combined with one or more *algorithms* expressed in Structured Text, Ladder Diagram, or Function Block Diagram (languages drawn from IEC 61131-3). A BFB transitions between

ECC states when input *events* arrive and associated Boolean guard conditions are satisfied. Executing an ECC state fires the associated algorithm, which may produce output events and set output data variables.

Composite Function Block (CFB) A network of interconnected FBs, providing hierarchical composition without a separate ECC. The internal topology is fixed at design time.

Service Interface Function Block (SIFB) Provides a typed interface to external services such as hardware peripherals, network protocols, or operating system calls.

The ECC of a BFB is structurally similar to an EFSM (Section 2.2): states, guarded transitions, and per-state algorithms. However, the trigger mechanism differs fundamentally — transitions fire only in response to named input *events* (discrete signals), not as a result of continuous condition polling or timer expiry. Timed behavior must be implemented using dedicated timer SIFBs wired into the network, and probabilistic transitions are not part of the standard.

3.2.3 4diac IDE and FORTE Runtime

The **4diac IDE** is based on the Eclipse framework. It provides a graphical editor for composing FB networks, an ECC editor for designing BFB state machines, and deployment tooling for distributing application partitions across multiple devices.

The **4diac FORTE** runtime is a portable C++ implementation of the IEC 61499 execution model targeting 16-bit and 32-bit embedded devices. It supports real-time execution and online reconfiguration of running applications. Supported platforms include Linux, Windows, VxWorks, and several bare-metal embedded targets.

3.2.4 Limitations Relevant to this Thesis

Despite its rigorous foundations, 4diac exhibits several limitations from the perspective of the Aetherium design goals:

- **Event-only trigger model.** The ECC transitions fire exclusively on named input events. There is no native support for time-based transitions (without auxiliary timer FBs), no probabilistic selection, and no direct reactivity to data threshold crossings without additional logic.
- **Scripting.** Algorithms are written in IEC 61131-3 languages (primarily Structured Text). There is no embedded general-purpose scripting language comparable to Lua. Dynamic behavior that depends on runtime arithmetic requires either pre-compiled C++ SIFBs or complex FB networks.
- **IDE weight.** The Eclipse-based IDE has a significant installation footprint and a steep learning curve, compared to a purpose-built Electron application.
- **No fault injection or replay.** 4diac provides no mechanism to inject network delays, packet loss, or disconnection events into a running deployment, and no execution trace suitable for step-by-step replay.

- **Execution-semantics complexity.** IEC 61499 execution semantics have been discussed extensively in the literature. The basic function block syntax and semantics were specified informally and ambiguously [41], motivating formal semantic models. While later editions and concrete runtimes such as FORTE define practical behavior, the event-driven ECC execution model remains more complex to reason about than the deterministic tick-based resolution used in Aetherium.

3.3 Comparison

Table 3.1 summarizes the key characteristics of both analyzed platforms against Aetherium Automata.

Table 3.1: Feature comparison of Node-RED, Eclipse 4diac, and Aetherium Automata.

Feature	Node-RED	4diac	Aetherium
Explicit state model	No	Yes (ECC)	Yes (EFSM)
Transition types	N/A	Event + guard	Classic, Timed, Event, Probabilistic, Immediate
Scripting language	JavaScript	Structured Text	Lua 5.4
Visual editor	Browser (Node.js)	Eclipse IDE	Electron desktop
Embedded runtime	No	Yes (FORTE, C++)	Yes (C++17)
Embedded targets	Raspberry Pi+	16/32-bit MCUs	ESP32, MCXN947
Program format	JSON	XML (FBT)	YAML
Typed variables	No (JavaScript)	Yes (IEC 61131-3)	Yes (7 types)
Fault injection	No	No	Yes
Time-travel debugging	No	No	Yes
Structural analysis	No	No	Petri net
Target domain	General IoT	Industrial	IoT + embedded

The analysis reveals that neither platform simultaneously satisfies all of the requirements identified in Section 3.4. Node-RED lacks explicit state modeling entirely and cannot run on bare-metal embedded targets. 4diac provides formal state machines through the ECC but restricts transitions to an event-driven model and provides no fault injection or replay capability. Aetherium Automata addresses these gaps by combining a five-type EFSM model, a portable C++ runtime, deterministic fault injection, and execution trace replay within a single unified toolchain.

Programmer's mental model. Beyond feature checklists, the three platforms differ fundamentally in *what the programmer sees and reasons about.*

In **Node-RED**, the primary abstraction is the *data flow*: messages travel between processing nodes along wires. The programmer thinks in terms of message routing — what arrives, what is transformed, where it is forwarded. State is implicit and must be maintained manually in the untyped context storage. There is no structural view of the system; the graph *is* the program, and scaling it produces a graph that grows in visual complexity without gaining analytical leverage.

In **Eclipse 4diac**, the programmer works at the level of *function block composition*: components are wired together at design time, and the per-component behavior is written as Structured Text algorithms attached to an ECC. The perspective is closer to traditional software engineering — the ECC is a design aid, but the live system is code executing inside a runtime that is largely opaque to the developer.

In **Aetherium**, the programmer operates at three complementary levels simultaneously. At the *behavioral level*, each device runs a named state machine whose current state, variable values, and pending transitions are live-visible in the IDE. At the *network level*, the IDE aggregates all connected devices and their signal exchanges into a single monitoring view. At the *structural level*, the Petri net panel lifts the entire multi-automata system into a higher-dimensional representation where dead states, bottleneck channels, and resource contention are structurally visible rather than guessed. This progression — from single-automaton behavior, through live network state, to structural network analysis — directly realizes the Deployment-Aware Development philosophy introduced in Section 1.1.

3.4 Requirements

Based on the analysis above, the thesis goals, and the two design principles introduced in Section 1.1, the following requirements were identified. Requirements R1–R4 form the foundational model and runtime; R5–R7 realize Reality-in-the-Loop; R8–R9 realize Deployment-Aware Development.

R1 – EFSM expressiveness The model must support guarded transitions, time-driven transitions, event/signal-driven transitions, probabilistic transitions, and unconditional (epsilon) transitions. Probabilistic transitions are essential for RitL: rare real-world events must be expressible and their frequency tunable without changing the automaton definition.

R2 – Scripting Guards and actions must be expressible as scripts evaluated against the current variable store. Lua is chosen as the scripting language.

R3 – Portability The runtime engine must compile and execute on desktop Linux/macOS, ESP32 [10] (Xtensa LX6, Arduino [2]/FreeRTOS [1]), and an ARM Cortex-M target. The NXP FRDM-MCXN947 evaluation board [30] (dual-core Cortex-M33) fulfills this role.

R4 – DSL Automata definitions must be authored in a human-readable format. YAML is chosen.

R5 – Real-time monitoring The IDE must display the current state, variable values, and connection metadata (RTT, connection type, and battery, if the device hardware exposes it) of running automata with low latency. *(DAD)*

R6 – Fault injection The system must support configurable fault injection (delay, jitter, drop, disconnect) at the communication boundary to enable controlled RitL testing. *(RitL)*

R7 – Time-travel debugging Execution traces must be recorded on the server and replayable step-by-step in the IDE, closing the observe-debug loop for injected faults. *(RitL)*

R8 – Structural analysis The system must lift multi-automata networks into Petri nets and surface dead states, bottleneck channels, and resource contention, providing the developer with a network-level structural view. *(DAD)*

R9 – Black-box contracts Automata must be able to declare a formal interface contract (ports, observable states, events) that is visible to monitors and to the fault injector without exposing internal implementation. *(RitL + DAD)*

Chapter 4

System Architecture

The Aetherium Automata platform is composed of four major components that form a layered architecture:

1. **Engine** — a portable C++17 library that parses, loads, and executes EFSM definitions.
2. **Server** — an Elixir/OTP application that manages connected devices, routes messages, and aggregates state.
3. **Gateway** — an Elixir/Phoenix application that exposes a WebSocket channel interface to the IDE.
4. **IDE** — an Electron desktop application (TypeScript/React) for visual design, deployment, and monitoring.

Figure 4.1 shows the high-level component topology.

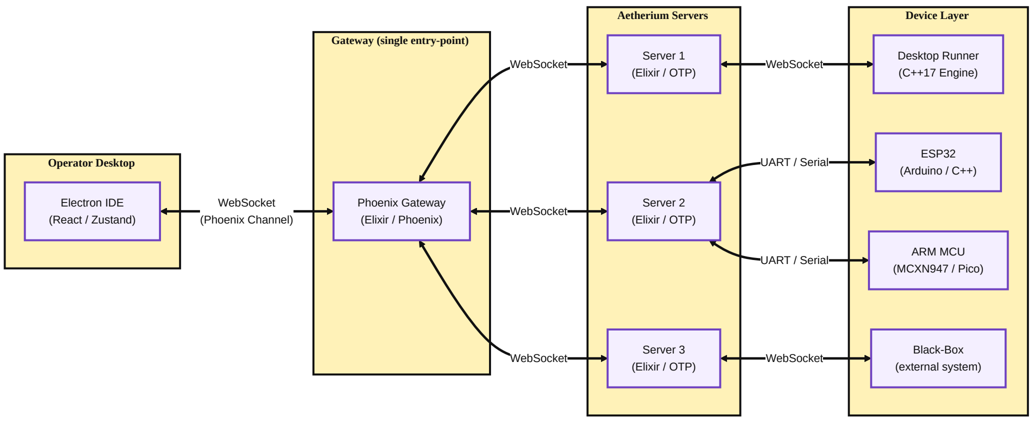

Figure 4.1: High-level component architecture of Aetherium Automata.

4.1 Technology Selection Rationale

Every major technology choice in Aetherium is a direct consequence of the two design principles introduced in Section 1.1. This section explains why each component uses the language and runtime that it does.

4.1.1 Elixir and the BEAM for the Server and Gateway

The server and gateway are written in Elixir [42], which compiles to bytecode for the BEAM virtual machine — the same runtime that underpins Erlang [3]. The choice is deliberate and has concrete implications for a fleet-management backend.

Process-per-device isolation. BEAM processes are not OS threads; they are lightweight VM-managed processes with small initial memory footprints that grow as needed. The current Erlang/OTP documentation states that a newly spawned process has a size of 327 words, including an initial heap area of 233 words [9]. In practice, this means that device and deployment work can be isolated into many supervised processes without the shared-memory overhead or lock contention that would accompany the same design in a thread-per-device model.

OTP supervision trees and let-it-crash. The Open Telecom Platform (OTP) provides a set of behavioral abstractions — GenServer, Supervisor, Registry — that impose a structured message-passing discipline on concurrent code. Every device process is supervised: if a device handler encounters an unexpected message or a protocol violation, it crashes and is restarted in isolation by its supervisor. The remaining devices are unaffected. This *let-it-crash* philosophy replaces defensive `try/catch` blocks with a structurally enforced recovery path, making the server resilient to individual device misbehavior by construction [3].

Preemptive scheduling and throughput. BEAM uses a preemptive reduction-counting scheduler: each process is allotted a fixed number of reductions (roughly equivalent to function calls) before it is descheduled. A single misbehaving handler cannot starve other processes, which is essential when one slow device must not delay telemetry from fifty healthy ones. The scheduler's cooperative multitasking analog in most IoT backends is the event loop, where a blocking call freezes the entire server. BEAM makes blocking impossible by design.

Automata hot deployment. Because device and deployment state are isolated inside supervised OTP processes, new automaton definitions can be deployed to a running target while other devices in the fleet continue executing uninterrupted. The deploy command starts from the YAML source model, validates it, and then selects a target-aware payload: desktop targets can accept YAML directly or through an `aeth_ir_v1` artifact wrapper, while constrained targets receive the compiled subset when their profile requires it. The engine on the device reloads the received automaton and restarts from the initial state. No server restart is required, and no unrelated device is affected.

4.1.2 C++ for the Engine

The engine is the component that runs on devices, spanning an extreme range: from a quad-core Linux desktop to a dual-core Xtensa LX6 (ESP32 [10]) or a dual-core ARM Cortex-M33 (NXP FRDM-MCXN947 [30]). The MCXN947 is the ARM Cortex-M representative in this thesis: it provides a tight-resource, bare-metal profile that validates the engine's portability to the class of microcontrollers where Raspberry Pi Pico and similar boards also live, with the added advantage that NXP's MCUXpresso SDK provides deterministic peripheral access without a third-party OS. This range rules out languages with managed runtimes or non-trivial standard library footprints. C++17 with disciplined allocator usage is the only mainstream language that can target all of these platforms with a single source tree.

Engine-as-library design. The engine is packaged as a static library rather than an application. The consuming project provides `main()`, hardware initialization, and the platform-specific implementations of three abstract interfaces. The engine itself contains no `main()`, no global state beyond what the application registers, and no compile-time assumption about the OS or threading model. This means the same engine code can be linked into:

- a desktop process that opens a WebSocket connection and runs the tick loop in a thread,
- an ESP32 Arduino [2] sketch that calls `engine.tick()` from its FreeRTOS [1] task,
- an MCXN947 bare-metal application that drives the tick loop from a hardware timer interrupt.

Abstract platform interfaces. The three interfaces that isolate platform-specific behavior are:

`IClock` Monotonic time source. Desktop builds delegate to the standard C++ steady clock, while embedded builds read a hardware timer. The interface returns a 64-bit microsecond count used for timed transitions and fault timing.

`IRandomSource` Pseudorandom source. Desktop builds use a 64-bit Mersenne Twister seeded from the host OS. Embedded builds use a hardware RNG when available, otherwise a compact software generator. Probabilistic transitions draw from this source on every evaluation.

`IScriptEngine` Script evaluation. Desktop builds use Lua 5.4 via Sol2 [40]. Embedded builds can substitute a lightweight evaluator or a no-op stub. With the stub, guards are reduced to static YAML values, resulting in a binary approximately 64 KB.

CMake build target selection. The top-level `CMakeLists.txt` [21] defines two interface targets: `aetherium_engine_desktop` and `aetherium_engine_embedded`. Linking against the former pulls in the Lua, IXWebSocket [23], and RapidYAML [5] dependencies. Linking against the latter pulls in only the embedded stubs. Platform-selection variables (`AETHERIUM_TARGET=esp32`, `AETHERIUM_TARGET=mcxn947`, `AETHERIUM_TARGET=desktop`) automatically configure the correct target.

4.2 Component Responsibilities

4.2.1 Engine

The engine is the core library. It is written in C++17 with no dynamic dependencies beyond the platform-specific backends (see Section 4.1). It is responsible for:

- parsing automata definitions from YAML [43] (via RapidYAML [5]),
- maintaining the variable store and evaluating Lua guards and actions (via Sol2 [40]),
- resolving and firing transitions on each execution tick,
- recording the execution trace for time-travel debugging,
- transmitting telemetry and receiving commands over the pluggable transport interface.

4.2.2 Server

The server is an Elixir/OTP application structured around a supervised process tree. It maintains device and deployment state through a central `DeviceManager`, supervised runtime processes, connector/session processes, and registries for addressable runtime instances. The stored metadata includes the last-known state snapshot, deployment status, connection metadata (transport type, RTT, and battery level if the device populates the protocol field), and outbound command state needed for acknowledged transfers. When a device disconnects, the deployment state remains available on the server so that a reconnecting target can continue along the same orchestration path, without forcing a server restart.

The server exposes an internal API consumed by the gateway. Incoming IDE commands (deploy, start, stop, set input, inject fault) are translated into OTP calls and casts against the appropriate manager, runtime, or connector process. Outgoing telemetry events are forwarded to a Phoenix PubSub topic to which the gateway subscribes.

4.2.3 Gateway

The gateway is an Elixir/Phoenix application that exposes a single *Phoenix Channel* endpoint at `/socket/websocket`. The IDE connects to this endpoint over a persistent WebSocket. The gateway acts as a protocol translator: it maps the IDE's JSON-over-WebSocket commands to internal server calls, and it relays the PubSub telemetry stream back to the IDE as structured channel events. The separation between the server (domain logic) and the gateway (transport/protocol) means that a REST API or CLI tool can consume the same server without touching the gateway.

4.2.4 IDE

The IDE is an Electron desktop application. The main process handles native file system access and exposes file I/O via IPC channels. The renderer process is a React single-page application using Zustand for state management. Communication with the gateway occurs exclusively through a `PhoenixGatewayService` singleton that manages connection lifecycle, channel joins, and automatic reconnection.

4.3 Component Interaction and Data Flow

Figure 4.1 shows the static topology; the dynamic interaction sequence is as follows.

1. The IDE loads a project file, and the developer clicks *Deploy*. The IDE sends the YAML source model as a `deploy` channel event to the gateway.

2. The gateway forwards the payload to the server, where the deployment compiler validates the model and chooses the correct target format. The resulting `LoadAutomata` payload is either YAML (`format=0x02`) or an `aeth_ir_v1` artifact (`format=0x01`) that may contain wrapped YAML or compact engine bytecode.

3. The server sends the binary frame to the device over its serial or WebSocket transport (embedded devices use UART serial; desktop emulators use a native WebSocket). Large payloads are split into acknowledged chunks.

4. The engine on the device receives the frame, reconstructs the payload if it was chunked, loads the YAML or IR artifact, and enters the *running* state. On every tick, it evaluates transitions and emits telemetry containing the current state, variable values, and timing metadata.

5. The server receives the telemetry frame, updates the deployment state, and publishes the event to the PubSub topic.

6. The gateway relays the event to all subscribed IDE channel sessions. The IDE updates the relevant Zustand store slice; connected React panels re-render automatically.

This pipeline is entirely event-driven: no polling occurs anywhere in the stack. The BEAM scheduler ensures that a burst of telemetry from one device does not delay messages for others.

4.4 Deployment Model

Devices are deployed using Docker Compose [6] for local multi-service orchestration. A typical local stack consists of the gateway service, the server service, one or more device emulator services (each running the engine as a desktop binary), and optionally a black-box probe container for sandboxed external systems. Real embedded targets (ESP32, MCXN947) connect to the server over a UART serial link; from the server's perspective, a physical device and a desktop emulator are indistinguishable—both speak the same binary wire protocol, differing only in the underlying transport carrier.

Chapter 5

Automata Model and DSL

5.1 Model Overview

An *automaton* in Aetherium is a named, versioned EFSM definition. At the top level, it contains:

- **config** — name, version, layout type (`inline` or `folder`), description, author, tags,
- **variables** — typed I/O and internal variables,
- **automata** — the EFSM body: initial state, states, and transitions.

5.2 States

Each state $q \in Q$ is identified by a string `StateId`: a letter or underscore followed by zero or more alphanumeric characters or underscores. States carry three optional Lua code hooks:

`on_enter` Executed once when the machine transitions into this state.

`body` Executed on every execution tick while the machine remains in this state.

`on_exit` Executed once immediately before transitioning away.

One state per automaton is designated the `initial_state`; the engine enters it at the start of execution.

5.3 Variables

Variables form the data context V of the EFSM. Each variable is defined by a *VariableSpec* that carries:

- **name** — unique string identifier,
- **type** — one of `bool`, `int32`, `int64`, `float32`, `float64`, `string`, `binary`, `table`,
- **direction** — `input` (read-only, written by the external environment), `output` (write-only for the environment, written by the automaton), or `internal` (read-write by the automaton),

- **default** — optional initial value.

At runtime, each variable is a *Variable* instance wrapping a type-safe `Value` variant and a boolean `changed()` flag. The flag is set on every write and cleared explicitly or on state transition. It is used by event-type transitions for O(1) reactive detection without polling.

5.4 Transition Types

Aetherium defines five transition kinds, each with distinct enabling semantics:

5.4.1 Classic

A classic transition is enabled when a Lua guard expression evaluates to `true`:

$$\text{enabled}(t) \iff g_t(V) = \mathit{true} \tag{5.1}$$

Guards are arbitrary Lua expressions operating on the variable store through the engine API.

5.4.2 Timed

A timed transition fires based on time domain conditions. Five modes are supported:

`after` Fire after a fixed delay Δt (in milliseconds) from the last state entry.

`every` Fire repeatedly with a period τ.

`timeout` Fire if no other transition from the same state has fired within Δt.

`at` Fire at an absolute timestamp.

`window` Fire only if the current time falls within $[t_{start}, t_{end}]$.

An optional jitter parameter $\varepsilon \geq 0$ adds a uniform random offset to the scheduled fire time:

$$t_{fire} = t_{target} + \mathcal{U}(0, \varepsilon) \tag{5.2}$$

This prevents synchronized firing across multiple automata instances and models real-world timer imprecision.

5.4.3 Event

An event transition is enabled when a named variable satisfies a specified signal condition. Supported trigger types are:

`on_change` Any write to the variable since the last evaluation.

`on_rise` The variable transitions from `false` to `true`.

`on_fall` The variable transitions from `true` to `false`.

`on_threshold` The variable crosses a comparator threshold: $v \bowtie k$ where $\bowtie \in \{>, <, \geq, \leq, =, \neq\}$.

An optional debounce delay δ_d filters spurious signals shorter than the threshold. The `one_shot` flag causes a threshold trigger to fire at most once per crossing. A transition may specify multiple triggers, combined with either AND or OR logic (`require_all`).

5.4.4 Probabilistic

A probabilistic transition is always structurally enabled. Its *weight* (represented as a fixed-point value between 0.00% and 100.00%) participates in a weighted roulette selection among all enabled transitions in the same priority group. The selection probability is:

$$P(T_i) = \frac{w_i}{\sum_{j \in G} w_j} \tag{5.3}$$

where G is the set of enabled transitions in the highest-priority group. Weights may be Lua expressions re-evaluated on each tick, enabling dynamic probability distributions.

5.4.5 Immediate (Epsilon)

An immediate transition is always enabled and resolved before any other type of transition. It models the ε-transitions of non-deterministic finite automata: unconditional, instantaneous state changes used to chain states without observable intermediate steps.

5.5 Transition Resolution Algorithm

On each execution tick, the engine resolves which transition, if any, fires from the current state. Algorithm 1 gives the full procedure.

Algorithm 1 Transition resolution on a single execution tick.

procedure TICK(q, $vars$)
 $T_{out} \leftarrow$ all outgoing transitions from state q
 $E \leftarrow \{t \in T_{out} \mid \text{Enabled}(t, vars)\}$ ▷ evaluate guards, timers, events
 if $E = \emptyset$ **then**
  execute `body` hook of q
  return
 end if
 $p^* \leftarrow \min_{t \in E} t.priority$
 $G \leftarrow \{t \in E \mid t.priority = p^*\}$ ▷ top-priority group
 if $|G| = 1$ **then**
  $t^* \leftarrow$ the sole element of G
 else if $\forall t \in G : t.weight > 0$ **then**
  $t^* \leftarrow \text{WeightedSample}(G)$ ▷ Eq. (5.3)
 else
  $t^* \leftarrow$ first element of G in declaration order
 end if
 execute `on_exit` hook of q
 execute `body` hook of t^*
 $q' \leftarrow \delta_E(q, t^*)$
 execute `on_enter` hook of q'
 return q'
end procedure

Priority establishes a total deterministic order; probabilistic selection applies only within a single priority group. This cleanly separates intentional non-determinism from deterministic control flow.

5.6 Black-Box Contract

An automaton may optionally declare a *black-box contract* — a formal interface specification for external systems. The contract defines:

- **ports** — named inputs/outputs with direction, type, observability flag (visible to monitors), and fault-injectability flag,
- **emitted events** — names of events the automaton produces,
- **observable states** — the subset of states visible to external monitors,
- **resources** — shared or exclusive resources with capacity and latency sensitivity annotations.

Black-box contracts are used by the structural analysis layer (Chapter 8) and by the fault injection subsystem.

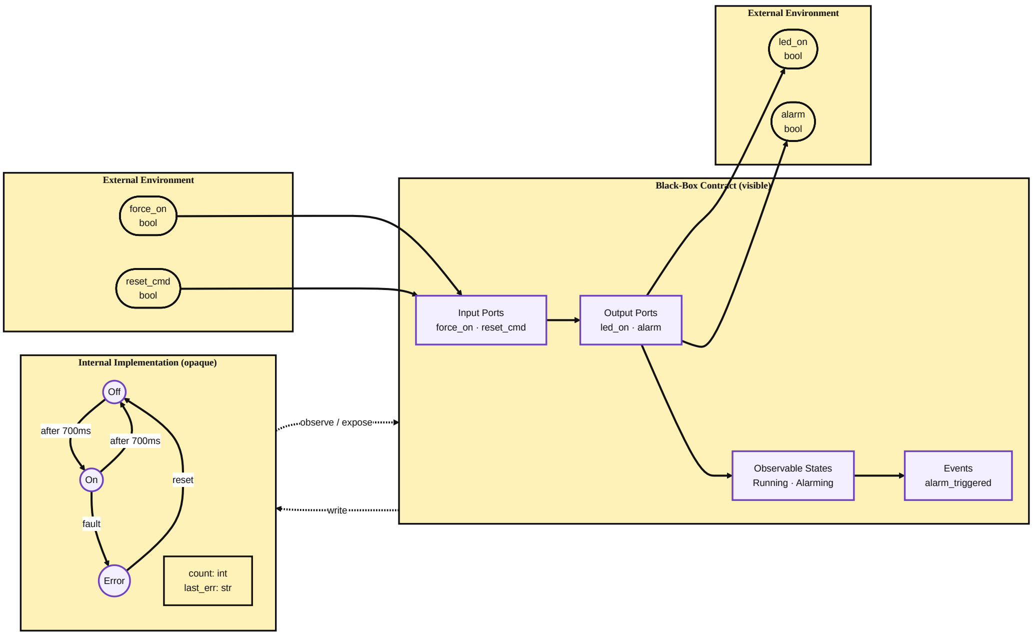

Figure 5.1: Black-box contract boundary. The visible contract layer exposes named input/output ports, observable states, and emitted events to the external environment. The internal implementation — states, variables, and transitions — is opaque.

5.7 YAML Domain-Specific Language

Automata are authored in YAML [43]. The top-level schema is:

```yaml
version: 0.0.1
config:
  name: <string>
  type: inline | folder
  location: <path> # required if type: folder
  language: lua # optional, default lua
  description: <string> # optional
  author: <string> # optional
  tags: [<string>, ...] # optional

variables:
  - name: <VarName>
    type: bool | int32 | int64 | float32 | float64 | string | binary
    direction: input | output | internal
    default: <value> # optional

automata:
  initial_state: <StateId>
  states:
    <StateId>:
      on_enter: |
        <lua code>
      body: |
        <lua code>
      on_exit: |
        <lua code>
  transitions:
    <TransitionId>:
      from: <StateId>
      to: <StateId>
      type: classic | timed | event | probabilistic | immediate
      priority: <int> # lower value = higher priority
      weight: <float> # for probabilistic selection
      condition: | # for type: classic
        <lua expression>
      timed: # for type: timed
        mode: after | at | every | timeout | window
        after: <ms>
        jitter: <ms> # optional
      event: # for type: event
        triggers:
          - signal: <VarName>
            trigger: on_change | on_rise | on_fall | on_threshold
            threshold:
              operator: ">" | "<" | ">=" | "<=" | "==" | "!="
              value: <number>
              one_shot: <bool>
        require_all: <bool>
```
Listing 5.1: YAML DSL top-level structure.

5.7.1 Layout Types

The `inline` layout places all state and transition Lua code directly inside the YAML file. The `folder` layout externalizes code into separate files referenced by path, enabling better IDE tooling, version-control diffs, and code reuse.

5.7.2 Validation Rules

The parser enforces the following constraints:

- All state and transition identifiers must follow the identifier naming convention (letter-or-underscore start, alphanumeric continuation).
- Every `from`/`to` endpoint must name a state defined in `automata.states`.
- Variable names must be unique within an automaton.
- Exactly one state must be designated `initial_state`.
- The `timed.mode` field determines which time parameters are required.

Chapter 6

Runtime Engine

6.1 Execution Tick Cycle

The engine runs a continuous tick loop. On each tick it advances timers, evaluates transition guards, selects and fires at most one transition per automaton, and emits telemetry. Figure 6.1 shows the full cycle.

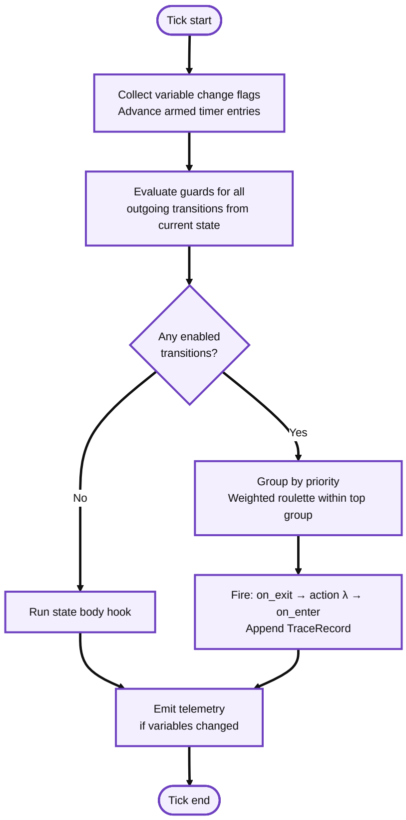

Figure 6.1: Engine execution tick cycle. A transition fires at most once per tick per automaton.

6.2 Platform Abstraction

All platform-specific behavior is isolated behind three abstract interfaces, allowing the same runtime core to compile on desktop Linux and bare-metal microcontrollers without `#ifdef` sprawl. Figure 6.2 shows the two-layer structure.

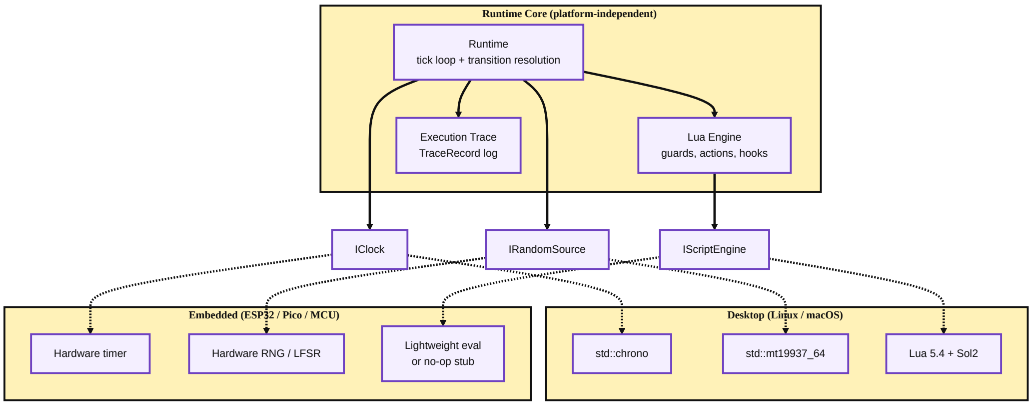

Figure 6.2: Platform abstraction layer. The runtime core depends only on the three interfaces; concrete implementations are selected at link time.

The runtime depends on three platform interfaces: clock, randomness, and script evaluation. On desktop, these bind to the standard C++ clock, a seeded PRNG, and Lua 5.4 [16] via Sol2 [40]. On constrained targets, the same interfaces can bind to hardware timers, compact random sources, and a lightweight evaluator or no-op script stub. CMake selects the concrete implementations at compile time; RapidYAML [5] and IXWebSocket [23] remain desktop-only dependencies.

6.3 Lua Scripting

Guards, state hooks (`onEnter`/`onExit`), and per-tick state code are all authored in Lua 5.4 [17, 16]. The engine's Lua environment exposes three API layers: variable I/O, hardware peripherals, and platform components. Together, they allow a single YAML automaton to control a real device's GPIOs, read sensors, drive displays, and stream live telemetry to the IDE without any C++ changes.

6.3.1 Variable I/O and Control API

The following globals are available in every script, on every platform:

`value(name)` Read the current runtime value of a named variable.

`setVal(name, v)` / `emit(name, v)` Write a value to an output variable; rejected at runtime if `name` is declared `input`.

`getInput(name)` Read an input variable; rejected if the variable is an output.

`setOutput(name, v)` Write an output variable with a stricter direction check than `setVal`.

`check(name)` / `changed(name)` Return `true` if the variable's change flag is set since the last tick — used to detect edge events.

`now()` Return the current monotonic timestamp in milliseconds (delegates to `IClock`).

`rand()` Return a uniform $[0, 1)$ float (delegates to `IRandomSource`).

`clamp(v, lo, hi)` Clamp v to $[lo, hi]$.

`log(level, msg)` Emit a structured log entry to the telemetry stream visible in the IDE Runtime panel.

`print(...)` Debug output; maps to `printf` on embedded, console on desktop.

6.3.2 Hardware Peripheral API

The engine registers four hardware tables in the Lua environment: `gpio`, `pwm`, `adc`, and `dac`. All calls pass through the `IHardwareService` abstraction; errors are thrown as Lua exceptions, so a misconfigured pin fails fast with an informative message rather than silently.

`gpio` — digital I/O.

`gpio.mode(pin, mode)` Configure a pin direction. Accepted values for `mode`: „`output"`, „`input"`, „`input_pullup"`, „`input_pulldown"`.

`gpio.write(pin, value)` Drive a pin high or low. Accepts a `bool` or an integer.

`gpio.read(pin)` Return the digital level of a pin as `0` or `1`.

The following `onEnter` snippet from the ESP32 follower showcase illustrates a typical use: the first line configures a status LED pin as output, and the PWM pin is bound with `pwm.attach`; thereafter, each tick drives both with a single write:

```
-- onEnter
gpio.mode(2, "output")
pwm.attach(0, 23, 5000, 8) -- channel 0, pin 23, 5 kHz, 8-bit

-- per-tick code
gpio.write(2, leader_button)
pwm.write(0, duty)
```

`pwm` — pulse-width modulation (ESP32).

`pwm.attach(channel, pin, freq, bits)` Bind an LEDC channel to a physical pin with the given frequency (Hz) and duty resolution (bits). On the ESP32, up to 8 channels can be active simultaneously [10].

`pwm.write(channel, duty)` Update the channel's duty cycle. For an 8-bit channel, the range is 0–255.

adc — analog-to-digital conversion (ESP32).

adc.read(pin) Return the raw ADC count (0–4095 for the ESP32 12-bit SAR ADC [10]).

adc.read_mv(pin) Return the calibrated millivolt reading, using the ESP-IDF `analogReadMilliVolts` calibration API [10].

The OLED dashboard showcase combines both calls to display a potentiometer reading in both raw counts and millivolts on the same screen refresh:

```
local pot = adc.read(34)
local pot_mv = adc.read_mv(34)
local duty = math.floor(clamp((pot / 4095.0) * 255, 0, 255))
pwm.write(0, duty)
```

dac — digital-to-analog conversion (ESP32 only).

dac.write(pin, value) Output an 8-bit analog voltage on pin 25 or 26 (the only DAC-capable pins on ESP32-WROOM).

i2c — two-wire bus.

i2c.open(bus, sda, scl [, freq]) Initialize an I^2C bus. `bus` = 0 maps to `Wire`, `bus` = 1 maps to `Wire1` on dual-bus ESP32 variants. Default frequency is 400 kHz.

i2c.scan([bus]) Scan all 127 I^2C addresses and return a Lua table of responding device addresses. Used in the IDE's *I2C scanner* component panel.

6.3.3 Component System

Complex peripherals are encapsulated as named *components* registered with `IHardwareService`. A script accesses a component by name via the `component(name)` global, which returns a Lua table whose methods map directly to the component's `invoke` handler:

```
local display = component("ssd1306_text")
display:init(128, 64, 0x3C, 0, 21, 22, 400000)
display:line(0, "Aetherium IDE")
display:line(1, "Pot: " .. tostring(pot))
```

The component system is open: any C++ class implementing the `IComponent` interface can be registered at startup. Currently shipped components are:

ssd1306_text (ESP32, implemented — not hardware-tested) SSD1306 128×64 OLED text renderer via Adafruit SSD1306. Methods: `init`, `clear`, `line(row, text)`, `show`, `set_text_size`, `invert`. The component manages the I^2C bus lifecycle internally; the script only provides display coordinates. Hardware validation was not performed because the display module was unavailable during development.

board_led (ESP32) Controls the built-in RGB/status LED. Methods: `on`, `off`, `set(bool)`, `status`.

i2c_scanner (ESP32) Wraps `i2c.scan` as a component so the IDE can trigger a scan via the Analyzer panel without any automaton code.

`board_temp` (MCXN947) Reads the on-chip temperature via the NXP LPADC sensor HAL. Methods: `init`, `read_c` (degrees Celsius), `read_milli_c` (milli-Celsius integer). The MCXN947 temperature guard showcase uses this to gate LED outputs on thermal thresholds.

`touch_pad` (MCXN947) Reads the TSI v6 capacitive touch sensor. Methods: `init([threshold])`, `raw`, `baseline`, `delta`, `pressed([threshold])`, `threshold`, `set_threshold`. Baseline calibration is performed on `init` by averaging eight consecutive scans; subsequent `pressed` checks subtract the baseline and compare to the threshold.

6.3.4 Network Connectivity

Network connectivity is handled at the *platform level*, not through Lua. The embedded engine communicates with the server over a UART serial link (`AetheriumEsp32SerialLink` / `AetheriumAvrSerialLink`); the MCXN947 target uses the same serial transport via `AetheriumMcxn947SerialLink`. The desktop emulator uses a native TCP/WebSocket connection via IXWebSocket. In all cases, the transport is transparent to the automaton: it sees only the variable inputs that arrive from the server and the variable outputs it emits back. No network or serial code ever appears in a Lua script.

6.3.5 Platform Capability Matrix

Table 6.1 summarizes which hardware APIs are available on each supported target.

Table 6.1: Lua hardware API availability by target platform.

API / Feature	Desktop	ESP32	MCXN947
`gpio.mode/write/read`	stub	✓ tested	✓ tested
`pwm.attach/write`	stub	✓ tested (LEDC)	—
`adc.read/read_mv`	stub	✓ tested (12-bit)	—
`dac.write`	stub	impl.	—
`i2c.open/scan`	stub	impl. (Wire/Wire1)	—
`ssd1306_text`	stub	impl., not tested	—
`board_temp`	—	—	✓ tested (LPADC)
`touch_pad`	—	—	✓ tested (TSI v6)
Transport	TCP/WS	Serial (UART)	Serial (UART)

Desktop stubs throw a descriptive error so that an automaton referencing `gpio.write` on the desktop emulator fails immediately with a clear message rather than silently succeeding with incorrect behavior.

On platforms where full Lua is unavailable (highly constrained microcontrollers), a no-op `IScriptEngine` stub can be substituted at compile time via the CMake target selection described in Section 4.1.

6.4 Timer Management

The runtime maintains a table of timer entries, one per timed transition. Each entry stores:

- `startTime` — the monotonic time at which the timer was armed,
- `targetTime` — the computed fire time (including jitter),
- `repeatCount` — remaining repetitions for `every` mode (0 = infinite).

On each tick, the runtime queries `IClock::now()` and compares it against `targetTime` for each armed timer. Expired timers with `mode = every` are immediately re-armed with a new `targetTime = now + period + jitter`.

6.5 Fault Injection

Fault injection is a proven technique for testing the resilience of distributed systems under realistic adverse conditions [4]. A `FaultProfile` can be attached to a deployment descriptor per device. The profile specifies:

Ingress faults Applied to messages arriving at the device:

- fixed delay and jitter (ms),
- drop probability $p_d \in [0, 1]$,
- duplicate probability $p_{dup} \in [0, 1]$.

Egress faults The same parameters are applied to outgoing messages.

Disconnect simulation Periodic forced disconnection windows of configurable duration.

Battery drain Per-tick and per-message energy consumption, combined with a battery profile that models charge state and cutoff voltage.

All random decisions within the fault injector use the platform `IRandomSource`, seeded by a field in the `FaultProfile`, ensuring deterministic, reproducible fault scenarios across runs.

6.6 Execution Trace and Time-Travel Debugging

Record-and-replay is a well-established strategy for debugging distributed systems: events are recorded during execution and can be replayed deterministically to reproduce failures [12]. Aetherium applies this principle at the automata level. Every engine event is recorded in an `ExecutionTrace` as a sequence of `TraceRecord` entries. Each record carries:

- sequence number,
- wall-clock and monotonic timestamps,
- event type (state transition, variable change, message send/receive),
- old and new values,
- latency measurements,
- the fault actions applied to the associated message (if any).

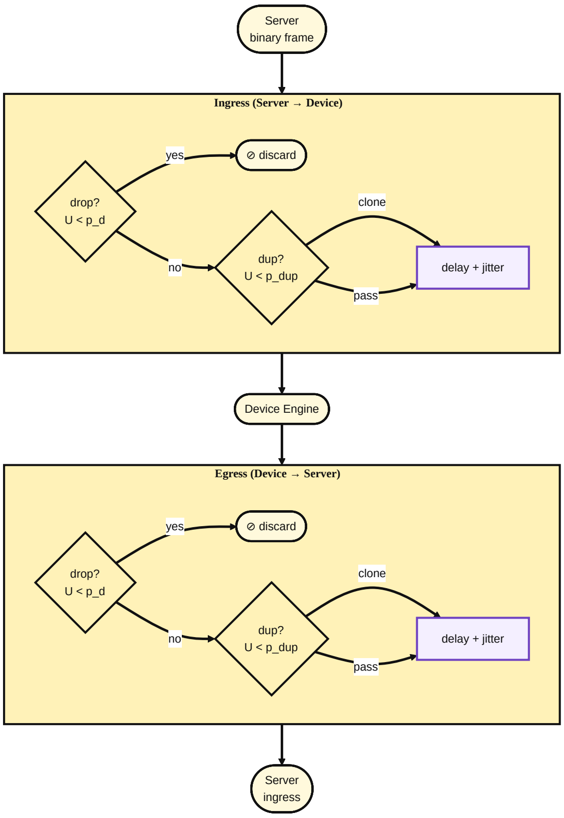

Figure 6.3: FaultProfile message pipeline. Each frame passes through drop, duplicate, and delay decisions independently for ingress (server → device) and egress (device → server). All probabilistic decisions draw from a seeded random source, making fault scenarios fully reproducible.

The engine accumulates trace records in a `LocalTraceStore`; the server persists them as a time-series in `time_series_store` and can optionally mirror them into InfluxDB, a database class intended for timestamped measurements and events [18]. The IDE's runtime view panel can step forward and backward through the server-reconstructed trace, retrieving the exact variable and state values at each recorded instant via `replay_state_at`. This enables deterministic replay of timing-sensitive bugs without requiring hardware reproduction of the fault.

Server-Side State Reconstruction and `RestoreState`

Stepping back in time in the IDE triggers a **full rewind** of the physical device, not merely a visual replay. The implementation spans three layers:

1. **Server-side time series.** An observability module records every `state_changed` and `variable_updated` event forwarded from the device into a per-deployment time series, together with periodic full-state snapshots. The `replay_state_at` function reconstructs the complete automaton state at any past timestamp by finding the nearest earlier snapshot and replaying the events that follow it.
2. **`RestoreState` protocol message (`0x48`).** A new message type was added to the Automata Plane of the binary wire protocol (range `0x40`–`0x7F`). The message carries the target state name and a snapshot of all variable values as a compact TLV-encoded list. On the Elixir side, `EngineProtocol.encode(:restore_state, %{...})` encodes the frame; `DeviceTransport.send_message/3` delivers it over the existing WebSocket transport.
3. **`Runtime::restoreState` in the C++ engine.** Upon receiving a `RestoreState` frame the engine handler pauses the tick loop, directly sets `ctx_.currentState` and applies the variable snapshot via `ctx_.variables.setValue`, cancels all pending timers, and leaves the engine in `Paused` state awaiting a `Resume` command. No `onEnter` hooks are fired — the state is restored atomically without side-effects.

The gateway channel `rewind_deployment` event orchestrates the flow: the server reconstructs the state at the requested timestamp, broadcasts a `deployment_status` update to connected IDE clients, then dispatches `RestoreState` to the physical device. Replay testing confirms that the runtime accepts the reconstructed state and, after an explicit `Resume`, continues with transitions consistent with the restored variable values.

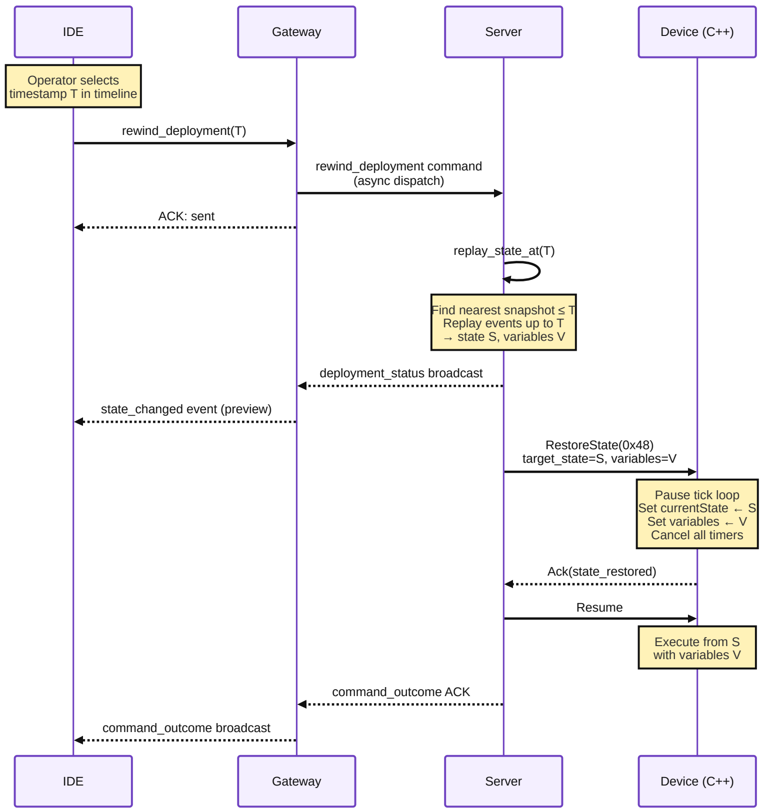

Figure 6.4: Time-travel rewind sequence. The IDE dispatches `rewind_deployment` to the gateway; the server reconstructs the automaton state at timestamp T via `replay_state_at` and delivers a `RestoreState(0x48)` binary command to the device, which pauses and applies the state snapshot atomically. Execution resumes through an explicit `Resume` command.

Chapter 7

Gateway and Communication Protocol

7.1 Binary Wire Protocol

All messages between the engine and the server use a compact binary envelope shown in Figure 7.1.

$$\underbrace{\texttt{0xAE01}}_{\substack{\text{platform tag}\\\text{(AE Automata v1)}}} \quad \underbrace{\texttt{0x01}}_{\substack{\text{protocol}\\\text{version}}} \; \underbrace{\texttt{type}}_{\text{1 byte}} \; \underbrace{\texttt{length}}_{\text{2 bytes}} \; \underbrace{\texttt{payload}}_{\text{N bytes}}$$

Figure 7.1: Binary message envelope. The 2-byte platform tag `0xAE01` identifies the Aetherium Automata protocol family (v1); the following protocol version byte allows the same platform to evolve its wire format independently.

The protocol is inspired by MQTT's [31] lightweight, topic-structured publish/subscribe model, but is implemented as a custom binary format rather than using the MQTT standard directly. The reason is architectural rather than a limitation of MQTT as a transport: Aetherium needs every device command and telemetry event to carry target identifiers, run identifiers, sequence information, deployment format metadata, and time-travel control messages in the same compact frame family. A generic MQTT deployment could carry similar information in topics and payloads, but Aetherium would still need an additional tracing and command schema layered on top. The custom envelope keeps that schema explicit. The length field is a 16-bit big-endian payload size, so a single frame carries at most 65 535 bytes before chunking is applied.

Messages are organized into three categories:

Control plane `Hello`, `HelloAck`, `Ping`, `Pong`, `Discover`, `Provision`, `Goodbye`, `Ack`, and `Nak` — connection management and reliable command progress.

Automata plane `LoadAutomata`, `LoadAck`, `Start`, `Stop`, `Pause`, `Resume`, `Reset`, and `RestoreState` — deployment, lifecycle, and time-travel commands.

Data plane `Variable`, `TransitionFired`, `Status`, `Debug`, and `Error` — runtime telemetry, diagnostics, and state-change evidence.

The `LoadAutomata` payload contains target and run metadata, a format byte, chunk metadata, and the automaton payload. The format byte distinguishes raw YAML (`0x02`) from `aeth_ir_v1` artifacts (`0x01`). This allows the same protocol to serve both development-friendly desktop targets that can still parse YAML and constrained targets that require server-side compilation. Large automata definitions or artifacts that exceed the frame size are fragmented using a chunked-transfer sub-protocol before being reassembled on the receiver side.

7.2 Phoenix Gateway

The gateway is implemented as an Elixir [42]/Phoenix [37] application using Phoenix Channels. Phoenix Channels provide persistent, topic-routed bidirectional WebSocket [11] connections with presence tracking. Each IDE client session corresponds to a single channel subscription.

The gateway exposes the following operations to the IDE:

- `ping()` — test channel connectivity,
- `listDevices()` — retrieve the current device registry,
- `deployAutomata(yaml, deviceId)` — transmit an automaton definition to a device,
- `startAutomata(deviceId, runId)` — command a device to begin execution,
- `stopAutomata(deviceId, runId)` — command a device to halt,
- `setInput(deviceId, varName, value)` — write an input variable,
- `getSnapshot()` — retrieve the current full device/variable state.

Snapshot caching reduces reconnection latency: when the IDE reconnects, it receives the last cached snapshot rather than waiting for a full device re-advertisement cycle.

7.3 End-to-End Message Flow

7.4 Integrated Development Environment

7.4.1 IDE Architecture

The IDE is built on Electron [35], which embeds a Node.js main process and a Chromium-based renderer process. The main process handles native file system access and IPC bridges. The renderer is a React [24] single-page application. The editor uses project-specific panels rather than a heavyweight embedded editor component — the primary operator workflow is driven by the graph canvas, property panels, gateway controls, and runtime monitor, with YAML kept directly editable for version-control-friendly changes.

State management uses Zustand [38], a lightweight library that exposes individual atomic stores with direct mutation. Each store covers a narrow concern:

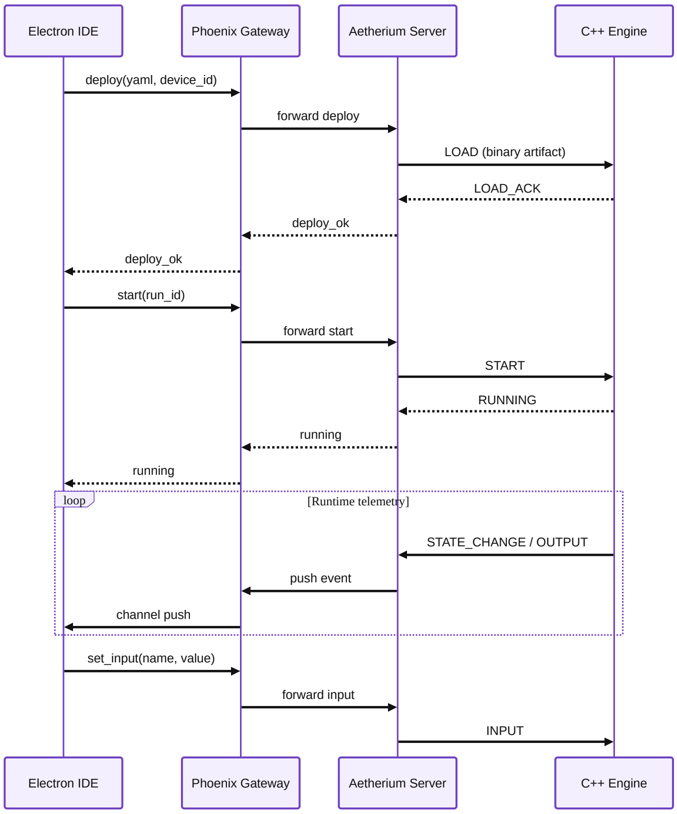

Figure 7.2: End-to-end message sequence for deploy, start, and runtime I/O exchange.

Table 7.1: Zustand stores in the IDE renderer.

Store	Responsibility
`automataStore`	Current automaton definition being edited
`projectStore`	Multi-automaton project and file management
`runtimeViewStore`	Live visualization state (current state, variable values, zoom)
`gatewayStore`	Phoenix channel connection, device list, pending commands
`analyzerStore`	Petri net analysis results (bottlenecks, contention)

7.4.2 Phoenix Gateway Service

`PhoenixGatewayService` is the IDE-side client for the Phoenix Channel. It manages the WebSocket lifecycle (connect, reconnect, presence), serializes commands into channel push events, and dispatches received events to the appropriate Zustand store actions. Command outcome tracking allows the UI to display pending/success/error states per operation.

7.4.3 Operator Panels

The IDE UI is organized into panels:

Canvas / State Machine Editor Visual graph editor. States are rendered as nodes; transitions as directed edges annotated with type, priority, and weight. Users can drag to create states, draw transitions, and edit properties inline.

State Inspector Edits `on_enter`, `body`, and `on_exit` Lua code for the selected state with syntax highlighting.

Transition Inspector Edits condition, type, priority, weight, and type-specific parameters (timed mode, event triggers) for the selected transition.

Variables Panel Unified view of all input, output, and internal variables with current values and types.

Gateway Panel Connection settings, device list, and manual command testing.

Runtime Monitor Panel Live display of the current state (highlighted in the canvas) and variable values during execution.

Log Panel Streaming telemetry event log with filtering by level and source.

Network Panel Topology view of the multi-device network and inter-automaton wiring.

Analyzer Panel Petri net analysis output: dead-state warnings, bottleneck candidates, contention reports.

Petri Net Panel Visual representation of the Petri net derived from the automata network.

Explorer Panel File-based project navigator for folder-layout automata.

The IDE therefore separates two editing modes. The textual mode keeps the YAML source available for precise inspection and version-control-friendly changes. The visual mode exposes the same model as states, transitions, variables, runtime status, and network topology. This split is important for the deployment flow: the IDE does not deploy a separate diagram format. It deploys the same YAML-backed model, which the server later compiles into either YAML or `aeth_ir_v1` payloads for the selected target.

Chapter 8

Petri Net Analysis

Petri nets [27] are a well-studied formalism for modeling and analyzing concurrent systems with shared resources. Their graphical nature and formal semantics make them well-suited as a structural analysis layer above a network of communicating automata. This chapter describes how Aetherium lifts an automata network into a Place/Transition (P/T) net, how individual nets are composed into a single model of the full system, the analysis operations the IDE exposes, and how two showcase scenarios demonstrate these capabilities.

8.1 Formal Definitions

Automaton. An Aetherium automaton is a tuple $\mathcal{A}_i = (S_i,\, s_i^0,\, T_i,\, V_i,\, G_i,\, Act_i,\, \Sigma_i^{\text{out}},\, \Sigma_i^{\text{in}})$ where:

- S_i is a finite set of states, $s_i^0 \in S_i$ is the initial state,
- $T_i \subseteq S_i \times S_i$ is the set of transitions,
- V_i is the variable store (typed name–value pairs),
- $G_i : T_i \to \mathit{Bool}$ is the guard function evaluated by the script engine,
- $Act_i : T_i \to \mathit{Void}$ is the action function executed on firing,
- Σ_i^{out} is the set of named output signal channels this automaton writes to,
- Σ_i^{in} is the set of named input signal channels this automaton reads from.

P/T net. A Place/Transition net is a tuple $\mathcal{N} = (P,\, T,\, F,\, W,\, M^0)$ [27] where P and T are disjoint finite sets of *places* and *transitions*, $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation, $W : F \to \mathbb{N}_{>0}$ assigns arc weights, and $M^0 : P \to \mathbb{N}_{\geq 0}$ is the initial marking.

A transition $t \in T$ is *enabled* under marking M iff for every pre-place p with $(p, t) \in F$: $M(p) \geq W(p, t)$. Firing t produces marking M' where $M'(p) = M(p) - W(p, t) + W(t, p)$, with W extended to return 0 for non-existent arcs.

8.2 Lifting a Single Automaton

The *lifting* of automaton $\mathcal{A}_i$ to a subnet $\mathcal{N}_i = (P_i,\, T_i',\, F_i,\, W_i,\, M_i^0)$ is defined as follows.

Places. Each state $s \in S_i$ becomes a place: $P_i = S_i$. Additionally, each output channel $\sigma \in \Sigma_i^{\text{out}}$ yields a dedicated *channel place* p_σ. The channel places are shared with the subnets of automata that list σ in their Σ^{in}.

Transitions and arcs. Each automaton transition $(s_a, s_b) \in T_i$ yields a Petri net transition t_{ab} with:

- a pre-arc (p_{s_a}, t_{ab}) consuming the token from the source state place,
- a post-arc (t_{ab}, p_{s_b}) depositing a token in the target state place.

If the automaton transition also writes to a signal channel $\sigma \in \Sigma_i^{\text{out}}$, an additional post-arc (t_{ab}, p_σ) is added. If the transition guard requires a token on an input channel $\sigma \in \Sigma_i^{\text{in}}$, an additional pre-arc (p_σ, t_{ab}) is added and the token is consumed when the transition fires.

All arc weights are 1 in this basic lifting. Bounded-capacity resources (declared in black-box contracts) are represented by mutual-exclusion places with initial marking equal to the resource capacity.

Initial marking. $M_i^0(s_i^0) = 1$ for the initial state; $M_i^0(s) = 0$ for all other state places; $M_i^0(p_\sigma) = 0$ for all channel places (channels start empty).

Figure 8.1 illustrates the construction for two automata communicating through a single named channel.

8.3 Network Composition

Given a project containing automata $\mathcal{A}_1, \ldots, \mathcal{A}_n$, the composed network net $\mathcal{N}$ is constructed by taking the disjoint union of all subnets and identifying shared channel places:

$$\mathcal{N} = \Big(\bigcup_i P_i \cup C,\ \bigcup_i T_i',\ \bigcup_i F_i,\ \ W,\ \ M^0 \Big)$$

where $C = \bigcup_i \Sigma_i^{\text{out}}$ is the set of channel places (each appearing once regardless of how many automata reference it), and M^0 is the union of the individual initial markings.

The composition is purely *structural*: it requires no execution trace, only the static YAML definitions. This means the analyzer can report structural hazards before any device is connected.

8.4 Analysis Operations

The analysis layer operates on the static Petri net graph derived from the YAML definitions. No execution trace is required; all checks run before any device is connected. Three structural observations are surfaced.

8.4.1 Dead-State Detection

A state place with no outgoing transition arcs is a structural terminal state. If the automaton declares no intentional final state, such a place is flagged as a structural dead end from which the automaton cannot make further progress. This check is linear in the number of places and does not enumerate markings. Full dynamic deadlock detection over

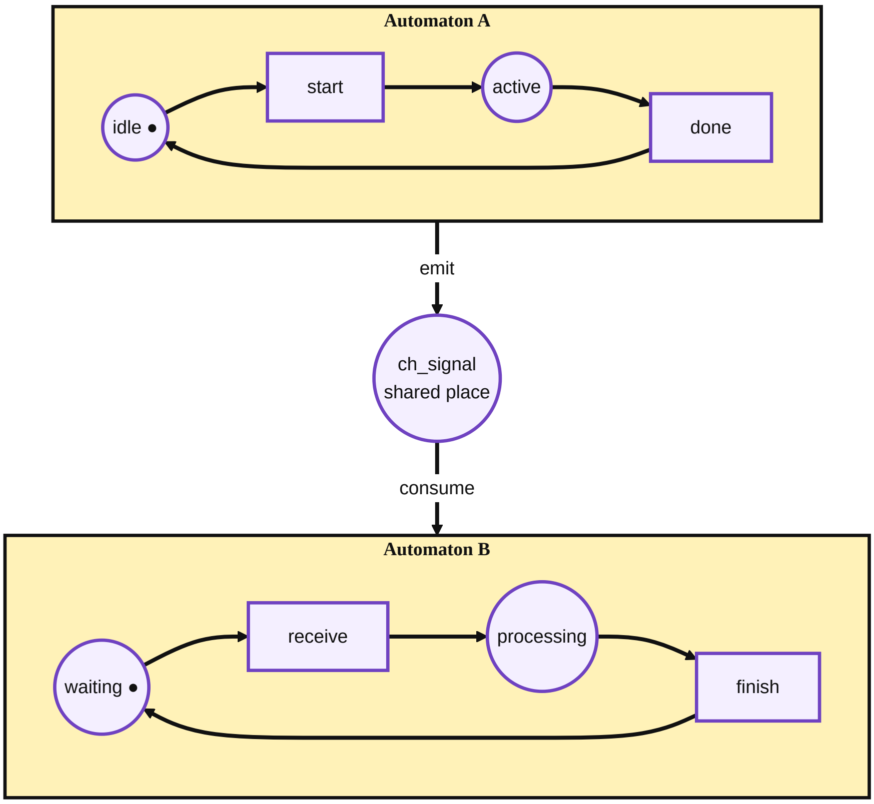

Figure 8.1: Two automata (A and B) lifted to Petri net subnets. The shared channel place `ch_signal` connects the output transition of automaton A to an input transition of automaton B.

the reachability graph — which requires enumerating all reachable marking tuples — is left as future work.

8.4.2 Structural Bottleneck Detection

A channel place whose number of input arcs exceeds its number of output arcs is a structural candidate for token accumulation. This indicates a design-time producer-consumer mismatch: one automaton may emit on the channel more frequently than the consumers drain it. The check counts arc cardinality on the composed graph rather than enumerating reachable markings; it provides a heuristic warning rather than a formal boundedness proof.

8.4.3 Resource Contention Identification

Resources declared in black-box contracts (Section 5.6) become shared resource places initialized with a token count equal to the declared capacity. Any two automata with arcs into the same resource place are reported as potential competitors for that resource. The report names the resource, its capacity, and the automata involved, giving the developer a concrete target for adding arbitration logic. If the sum of the static per-automaton resource demands exceeds the declared capacity, a capacity warning is issued.

8.5 Showcase Scenarios

Two showcase categories directly exercise the Petri net analysis.

8.5.1 Showcase 13 — Petri Signal Chain

The signal chain consists of four automata arranged as a linear pipeline:

$$\mathit{CommandRouter} \longrightarrow \mathit{SafetyGate} \longrightarrow \mathit{DriveUnit} \longrightarrow \mathit{TelemetryObserver}$$

Each automaton reads from one channel place and writes to the next. After lifting and composition, the result is a four-subnet net with three channel places. Each subnet has exactly one state place holding a token at any time — a property that follows directly from the lifting construction: each automaton starts with one token in its initial state, and each transition moves that token forward. The safety gate contains a *locked* state that has no outgoing arc to the downstream channel place. This is structurally visible in the Petri net: the channel place between the gate and the drive unit has more input arcs than output arcs, which the structural bottleneck check flags as a producer-consumer mismatch. The showcase is designed so that this bottleneck is detectable from the static YAML graph without requiring fault injection.

8.5.2 Showcase 14 — Petri Contention

Three automata compete for a shared power budget modeled as a resource place with capacity 2 (two power units available):

- **ChargerNode** requests 1 unit when active,
- **MotionAxis** requests 1 unit when moving,

- **PowerAllocator** requests 1 unit during peak arbitration.

The composed Petri net identifies all three automata as sharing the power-budget resource place. Each automaton declares a demand of 1 unit, giving a worst-case simultaneous demand of 3 units against a declared capacity of 2. The capacity warning names all three automata, giving the developer a clear target for adding a priority scheme or mutual-exclusion transitions to the power allocator.

Chapter 9

Showcase Catalog

The system is validated by a showcase set of 39 YAML automata organized into 15 categories. A curated validation catalog lists the desktop-runnable scenarios used for regression checks. All definitions reside under `example/automata/showcase/`.

Table 9.1: Showcase automata catalog.

#	Category	Key concepts demonstrated
01	Basics	Timed and classic transitions, manual override input
02	Control	Event threshold triggers, pump state control
03	Probabilistic	Weighted 3-lane quality routing
04	Resilience	Watchdog timer, fault detection, retry with backoff
05	Energy	Load prediction, peak shaving scheduler
06	Pipeline	Multi-stage line buffer dispatcher
07	Folderized	Folder-based code layout, door safety controller
08	ESP32	PWM, LED, OLED, thermostat, production line (8 automata)
09	MCXN947	GPIO, buttons, touch, temperature on ARM Cortex-M33 (4 automata)
10	Guarded Cell	6-automata safety supervision network
11	Bidir. Loop	4 automata with bidirectional signal wiring
12	Black Box	Docker-sandboxed probe, external system wrapping
13	Petri Sig. Chain	Command router → safety gate → drive unit → telemetry
14	Petri Contention	Charger, motion axis, power allocator competing for resources
15	Aetherium Gem	TDD checkpoints, high state churn, fault injection, replay markers, black-box contract, Petri-liftable resource demand

9.1 Flagship Showcase: Aetherium Gem Cell (15)

The Aetherium Gem Cell is the primary thesis demonstration project. It is intentionally state-heavy: the scenario contains boot, test, request, grant, processing, quality, rework, fault, isolation, replay, release, and completion phases. Most states execute only short assignments to observable variables such as `phase`, `demand_bus`, `heartbeat`, `tdd_step`, and `replay_marker`. This keeps behavior visible in the automaton structure rather than hiding it in long scripts.

The showcase combines the main platform abilities in one readable model:

- **TDD checkpoints**: three self-test states encode an arrange-act-assert flow and expose a validation result for automated checks.
- **Fault injection**: drop and delay inputs drive explicit fault and compensation paths, so the Gateway panel perturbs the same boundary that production uses.
- **Petri lift**: the workcell bus resource and demand output create a shared-resource place when the automaton is lifted into the Petri net view.
- **Time-travel debugging**: each major phase writes a replay marker, making the execution trace easy to inspect and rewind during demonstrations.
- **Black-box contract**: the model declares observable ports, emitted events, and resource metadata so the analyzer can reason about the boundary even if the internal implementation is hidden.

```
SelfTestAssert:
  on_enter: |
    setVal("phase", "tdd_assert")
    setVal("tdd_step", 3)
    setVal("tdd_assertion_passed", value("sensor_ok") == true)

request_fault_drop:
  from: RequestBus
  to: FaultInjected
  type: classic
  priority: 0
  condition: value("fault_inject_drop") == true or value("sensor_ok") == false
```

Listing 9.1: Excerpt from the Aetherium Gem Cell showcase.

The corresponding IDE project is the backend capabilities tour in the demo projects directory. It places the Gem Cell first, followed by the signal-chain, guarded-cell, power-contention, and watchdog networks. This ordering makes the project suitable for demonstrations: start with one comprehensive automaton, then show how the same ideas scale to a multi-automata network.

9.2 Selected Showcase: Quality Router (03)

The quality router demonstrates probabilistic transition selection. Three output lanes (A, B, C) are represented as transitions from a single routing state. Each transition carries a weight:

- Lane A: weight 5000 (50%)
- Lane B: weight 3000 (30%)
- Lane C: weight 2000 (20%)

On each tick, the transition resolver applies Equation (5.3) to select the lane. The weights are static in this example, but could be Lua expressions that respond to sensor readings (e.g., routing more batches to Lane A when Lane B is congested).

9.3 Selected Showcase: Sensor Watchdog Recovery (04)

The watchdog demonstrates timed transitions combined with classic guarded recovery. Figure 9.1 shows the full state machine.

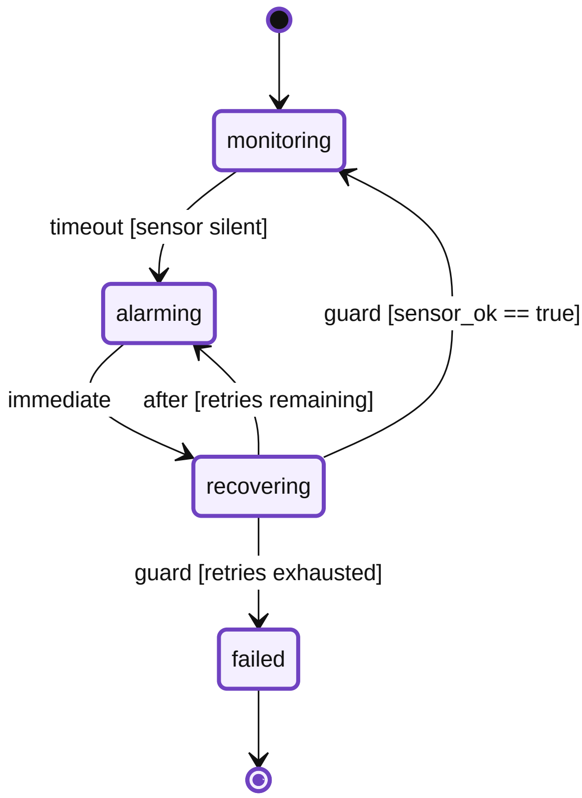

Figure 9.1: Sensor watchdog automaton. Dashed transitions are timed; solid transitions are classic guards. The `failed` state is a terminal escalation sink.

The automaton uses three transition types together: a *timed/timeout* transition detects sensor silence, *immediate* transitions chain states without observable intermediate steps, and *classic* guards check the `retry_count` variable to decide between another attempt and escalation to failure. This pattern recurs in many real-world IoT deployments: detect device failure, raise an alarm, attempt structured recovery, and escalate definitively if recovery is exhausted.

9.4 Catalog as a Validation Corpus

The showcase catalog is not only presentation material. It acts as a regression corpus for the YAML DSL and for the runtime semantics. This is why the examples are intentionally varied rather than all being minor variations of a blinking LED. A good validation corpus must contain small cases that isolate individual features and larger cases that combine features in ways that resemble real deployments.

The first categories are deliberately narrow. The basics and control examples validate that ordinary timed, event, and guarded transitions can be expressed cleanly. The probabilistic quality router isolates weighted transition selection. The watchdog example isolates failure detection and recovery escalation. These small models are easy to inspect manually and are useful when a parser or runtime regression must be localized quickly.

The middle categories introduce composition pressure. Pipeline and bidirectional loop examples demonstrate that signals can flow through multiple automata without losing their intended direction. The guarded-cell network validates supervision logic across several cooperating automata. The ESP32 and MCXN947 categories demonstrate that the same model can target hardware-specific bindings without changing the conceptual EFSM structure.

The latter categories target the thesis-specific features. Black-box examples validate that an opaque component can participate through an explicit signal contract. Petri signal-chain and contention examples demonstrate that structural analysis identifies bottlenecks and shared-resource contention. The Aetherium Gem Cell intentionally combines TDD checkpoints, fault-injection inputs, replay markers, state churn, and Petri-liftable resource demand into a single flagship model.

This distribution gives the catalog two roles. During development, it serves as a quick regression suite: the curated `CATALOG.txt` set should validate after every substantial change. During the presentation, it is a guided tour: the reader can start from one-feature examples and progress toward the Gem Cell without learning a new notation at each step.

9.5 Demonstration Selection Criteria

Only a subset of the catalog is used in the thesis walkthrough because a thesis demonstration must be both representative and readable. The selected examples were chosen according to four criteria:

1. The example must demonstrate a capability central to the thesis goals.
2. The state graph must be readable in the IDE without excessive zooming or hidden detail.
3. The runtime behavior must produce observable state or variable changes within a short demonstration window.
4. The example must either validate a single feature clearly or combine multiple features in a way that justifies the added complexity.

The Gem Cell satisfies the fourth criterion in the strongest way: it is complex enough to show that the platform is not limited to toy automata, while keeping code blocks short so that state transitions remain the primary explanation mechanism. The watchdog satisfies the opposite role: it is small, readable, and immediately understandable as a real control pattern. The Petri signal-chain and contention scenarios then show why network-level analysis matters once several individually simple automata interact.

This selection keeps the demonstration focused. Rather than presenting all 39 automata individually, the thesis uses the catalog as supporting evidence and focuses on the scenarios that best demonstrate the platform's distinctive capabilities.

Chapter 10

Testing and Validation

Validation covers several layers, each targeting a different failure mode. Unit and component tests confirm isolated behavior within parsers, the runtime, the protocol layer, and the stores. Integration tests verify contracts at process boundaries. End-to-end stack checks exercise the full gateway-server-device path under real conditions. The showcase catalog serves as a regression corpus for DSL and runtime features. Manual operator workflows cover the IDE and hardware paths where automated checks cannot reach.

10.1 Validation Scope

Figure 10.1 shows the validation coverage model used in the project. The model is intentionally layered: low-level tests verify deterministic behavior within a single component, while higher layers ensure that the same behavior remains correct when messages cross process, transport, and operator-interface boundaries.

The concrete validation layers are:

Engine build. `cmake -build build -target aetherium_engine` compiles the portable C++ runtime and command path used by desktop devices.

CLI and parser checks. `pytest` verifies command-line argument handling, YAML validation, trace export behavior, and deterministic error reporting when the selected desktop binary lacks a required loader.

Gateway tests. `mix test` in the gateway application covers authentication, registry behavior, command envelopes, protocol encoding, server channels, heartbeat handling, and variable updates.

Server tests. `mix test` in the server application covers automata runtime behavior, deployment compilation, device manager commands, time-series replay, analyzer projection, protocol handling, ROS2 connector logic, and showcase catalog loading.

IDE validation. `npm run typecheck` validates the TypeScript/Electron client. TypeScript is used here specifically as a static type checker that can detect type-shape errors before execution [25]. The current IDE package does not define a separate `npm test` target, so visual and command-flow behavior is covered by manual workflows.

Catalog checks. The showcase validation script checks the curated desktop-runnable set listed in the showcase catalog.

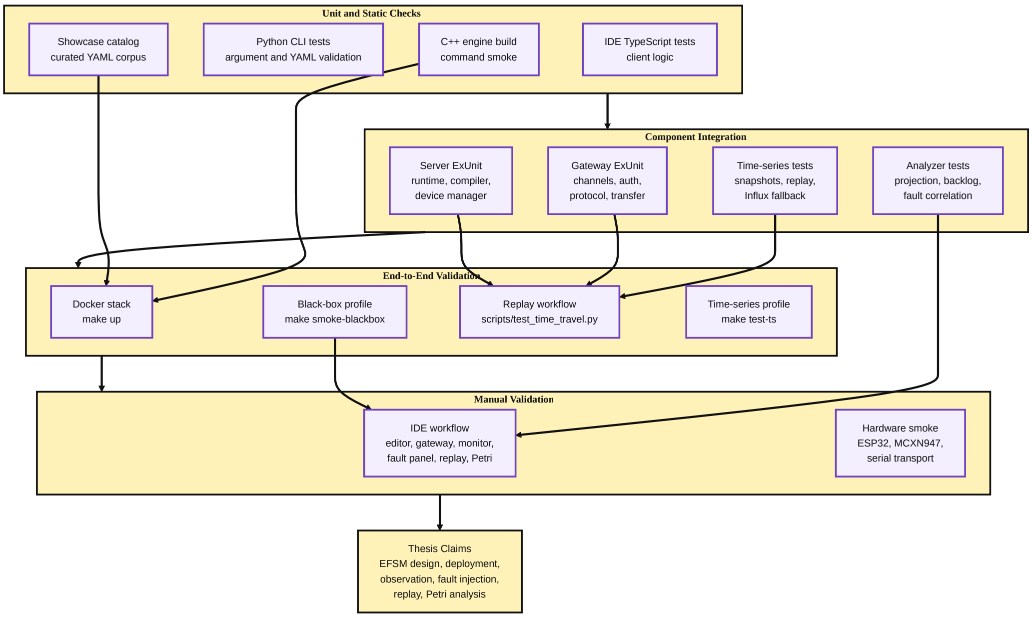

Figure 10.1: Validation coverage model. Unit tests cover isolated parsing, runtime, protocol, and store behavior; integration tests cover server/gateway/device boundaries; end-to-end tests exercise deployment, fault injection, and rewind; manual validation confirms the IDE and hardware workflows.

Runtime stack checks. The stack smoke targets exercise deployed services, black-box devices, and optional Influx time-series integration.

Manual workflow checks. IDE and hardware/container checklists confirm that visual deployment, monitoring, fault injection, replay, and Petri analysis remain usable from the operator surface.

Each layer targets a different risk: parser regressions, protocol incompatibility, runtime execution errors, gateway/server orchestration faults, IDE command wiring mistakes, and failures that only appear when several services run together. The suite is therefore useful both as proof of implemented behavior and as a regression detector: a failing layer identifies the boundary that no longer matches the implemented contract.

10.2 Requirement Traceability

Table 10.1 maps each requirement from Section 3.4 to the validation evidence that covers it. This is a feature-level traceability map, not a line-coverage report.

This mapping also makes missing evidence visible. For example, the hardware rows are validated through smoke tests and manual checks rather than exhaustive endurance testing. The IDE row is similarly split: static TypeScript correctness is automatically checked, while visual workflow correctness is manually validated because it depends on layout, operator feedback, and live gateway interaction.

Table 10.1: Requirement-to-validation traceability.

Requirement area	Validation evidence
Explicit EFSM model and YAML DSL	Python CLI checks, engine validation, showcase catalog validation, server YAML tests.
Multiple transition types	Runtime ExUnit tests, curated showcase cases for timed, event, probabilistic, immediate, and classic transitions.
Portable execution	CMake build, host-runtime deployment tests, target-profile compiler tests for desktop, AVR, ESP32, and MCXN947.
Real-time gateway communication	Gateway protocol tests, server channel tests, heartbeat tests, deployment transfer tests, Docker stack smoke tests.
Live IDE monitoring	Manual IDE checklist, Phoenix gateway service behavior, runtime monitor workflow in the demonstration.
Fault injection	Gateway fault workflow, device manager tests with fault metadata, manual delay/drop/disconnect profiles.
Time-travel debugging	Time-series store/query tests, replay script checks, rewind deployment command tests.
Petri net analysis	Analyzer projection tests, Petri signal-chain and contention showcases, manual Analyzer/Petri panel checks.
Black-box integration	Gateway black-box tests, device manager command validation, black-box stack smoke tests.
Hardware feasibility	ESP32 and MCXN947 build/flash smoke workflows, board-specific showcase definitions.

10.3 Unit and Component Tests

The C++ runtime is validated first by compilation and command smoke checks. These tests protect the lowest layer: if the runtime cannot parse arguments, validate YAML, export traces, or construct the command path, higher-level gateway and IDE tests would only report secondary failures. The Python test suite under `tests/` exercises this command-line surface and also detects build/profile mismatches, such as accidentally running a runtime-core-only binary where the YAML file loader is required.

The gateway and server use Elixir's ExUnit framework. Gateway tests cover the boundary between IDE clients, servers, and devices: token authentication, automata registration, command envelope normalization, binary protocol encoding, heartbeat handling, deployment transfer, connector status, and variable update propagation. These tests are intentionally close to the Phoenix channel boundary because most gateway regressions appear as malformed or misrouted messages rather than pure calculation errors.

Server tests cover the central orchestration behavior. The `automata_runtime` suite verifies state initialization, variable defaults, lifecycle commands, transition firing, event transitions, and runtime code hooks. The deployment compiler tests check target-specific compilation decisions for desktop, AVR, ESP32, and MCXN947 profiles, including the distinction between YAML-capable targets and targets that require `aeth_ir_v1`. Device manager tests exercise command handling, deployment rewind from recorded snapshots, host runtime execution through the standard deployment API, black-box command validation, topic-versioned input propagation, and error metadata captured during remote deployment failures.

Time-series and analyzer behavior are tested separately because they support the time-travel and Petri-analysis claims. The time-series store and query tests verify event/snapshot persistence, state reconstruction at a target timestamp, local backend selection, Influx backend selection, and fallback behavior. Analyzer projection tests verify that deployment evidence, connection backlog, blocked handoffs, queue pressure, latency findings, and injected-fault metadata are converted into structured analysis results instead of remaining as uncorrelated log entries.

10.4 Coverage by Component

The validation suite is organized around component ownership so that a failing test points directly at the layer that broke. Each component has a focused set of tests:

Runtime engine. Build and CLI tests cover startup, arguments, YAML validation, trace export, and command-smoke behavior. These tests establish that the lowest layer produces deterministic acceptance and rejection results before deployment is involved.

Server runtime. ExUnit tests cover the in-memory automata runtime: state initialization, `on_enter`/`body`/`on_exit` hooks, input updates, event transitions, weighted transitions, lifecycle commands, and host-runtime deployment paths.

Deployment compiler. Tests cover target-profile decisions. Rich desktop targets can accept YAML-oriented automata, while constrained profiles either compile supported subsets to `aeth_ir_v1` artifacts or reject unsupported constructs with deterministic errors.

Gateway. Tests cover authentication, server and device channels, command envelope normalization, heartbeat handling, deployment transfer, variable updates, and protocol compatibility between JSON channel messages and binary device messages.

Time-series layer. Store and query tests cover snapshot/event persistence, replay from local storage, Influx-backed replay, and fallback behavior when the Influx backend is unavailable.

Analyzer. Projection tests cover structural and evidence-based findings: queue backlog, blocked handoff, connection backlog, latency warnings, and injected-fault correlations.

IDE. The configured automated check is TypeScript validation via `npm run typecheck`. The remaining coverage is manual because the most important IDE behavior is visual workflow correctness across the editor, gateway, runtime monitor, analyzer, and Petri panels.

A parser failure should fail fast in Python or engine validation; a channel schema break should fail in gateway/server tests; a TypeScript contract failure should fail in the IDE typecheck; and a visual wiring problem should be caught by the manual operator checklist. No single „run the demo“ test is expected to cover all of this.

10.5 End-to-End Tests

End-to-end testing validates paths that cross process boundaries. The project includes a dedicated Mix task, `mix aetherium.e2e`, which deploys a flagship showcase target through the gateway/server path, runs a scripted input sequence, and checks that the resulting device state and emitted telemetry match the expected behavior. This is the highest-value automated check because it exercises the same protocol and deployment path used by the IDE.

The time-travel workflow is covered by `scripts/test_time_travel.py`. The script deploys a representative automaton, waits for state changes, requests a rewind of the deployment to an earlier timestamp, and verifies that the reconstructed state is accepted by the runtime. This test specifically guards the contract between the server's time-series query layer and the engine's `RestoreState` command; resuming execution after the restore remains an explicit operator or test action.

The Docker-based [6] smoke tests validate service composition. The core stack check starts the gateway, server, and C++ device container. The black-box smoke test starts the black-box profile and verifies activation/response behavior through the declared signal contract. The time-series integration test starts the Influx/Grafana profile and runs the server-side Influx integration suite with `RUN_INFLUX_INTEGRATION_TESTS=1`; in this stack, Grafana is used as the dashboard surface for longer-running metric views [13]. These checks are slower than unit tests but catch configuration, networking, and service-startup errors that isolated component tests cannot expose.

10.6 Fault Injection and Replay Validation

Fault injection and time-travel debugging are core to Reality-in-the-Loop, so they require validation beyond normal deployment success. The tested path begins with an ordinary automaton deployment, then deliberately perturbs the system boundary. A delay fault validates that the runtime continues ticking while transport latency changes. A drop fault validates that the analyzer and runtime surfaces report missing or delayed evidence instead of silently assuming nominal communication. A disconnect fault validates recovery behavior after a WebSocket session interruption.

The replay validation then checks the inverse operation: after enough events and snapshots have been recorded, the server reconstructs an earlier state and sends `RestoreState` to the engine. The expected result is not merely that the command succeeds. After the operator or test resumes execution, the next observed state transition must be consistent with the restored state and variable values. This condition is stronger than checking a response code because it validates the semantic link between historical reconstruction and live execution.

The Aetherium Gem Cell showcase supports this validation particularly well. Its replay-marker variable records named phase checkpoints, while the TDD step and assertion flag expose the arrange-act-assert self-test path. During a replay run, these values provide human-readable anchors in the Runtime Monitor and machine-readable evidence in the trace. The same automaton therefore supports automated assertions and manual inspection without maintaining two separate models.

10.7 Showcase Catalog Validation

The showcase catalog is both a demonstration asset and a regression corpus. Each curated YAML file checks a different slice of the DSL and runtime:

- timed transitions and manual override inputs,

- event thresholds and stateful control,
- probabilistic routing,
- watchdog recovery and retry escalation,
- pipeline dispatch,
- bidirectional signal wiring,
- black-box contracts,
- Petri signal-chain and contention models,
- the Aetherium Gem Cell with TDD checkpoints, fault-injection inputs, replay markers, and Petri-liftable resource demand.

The validation script reads `example/automata/showcase/CATALOG.txt` and validates the curated set against the engine's accepted YAML subset. This is deliberately narrower than „all examples in the repository“: some examples target specific hardware or documentation scenarios and are not expected to be desktop-runnable. Keeping a curated catalog makes regression results stable while still covering the core language features.

10.8 Manual Validation Protocol

Manual validation is treated as a protocol, not as casual clicking through the application. For each manual pass, the operator records the stack profile used, the selected automaton, the target device, the induced fault profile, if any, and the observed final state. This is important because manual validation is the only layer that checks visual correctness: whether the current state highlight appears on the expected node, whether the variable table updates without stale values, whether the Gateway panel reports the correct connection state, and whether the Petri graph is readable after layout.

The manual protocol uses three representative workflows:

1. **Nominal deployment workflow.** Start the core Docker stack, open the demo project, deploy the Gem Cell or watchdog automaton to the containerized C++ device, start execution, and verify live state and variable updates.
2. **Reality-in-the-Loop workflow.** Apply a seeded delay/drop/disconnect profile, observe the runtime/analyzer response, rewind to a pre-fault event, resume execution, and verify that the same phase sequence is reproduced.
3. **Structural-analysis workflow.** Open the Petri signal-chain, contention, and Gem Cell networks, run the analyzer, and verify that reported bottlenecks or resource demands correspond to the visible automata states and signal channels.

These workflows are deliberately aligned with the screenshots required by the thesis. A screenshot is considered valid only if it was captured during one of these workflows and shows a real, deployed, or loaded project state.

10.9 Manual IDE and Hardware Validation

Some requirements are inherently operator-facing and cannot be fully validated by headless tests. Manual validation, therefore, focuses on workflows that must remain understandable in the IDE:

1. Open `NewProject.aeth` and verify that the network list, explorer, and automata editor load the curated demo project.
2. Deploy the Aetherium Gem Cell or watchdog automaton to the containerized C++ target and confirm that live state, variables, and transition history update in the Runtime Monitor.
3. Apply `delay`, `drop`, or `disconnect` fault profiles in the Gateway panel and confirm that the runtime and analyzer report the induced condition.
4. Select a historical runtime event and execute the rewind workflow, verifying that the server reconstructs the state and the device can be resumed from the restored point.
5. Open the Petri Net and Analyzer panels for the Petri signal-chain, contention, and Gem Cell networks and verify that bottlenecks, resource demand, and anomaly summaries are displayed.
6. For ESP32 and MCXN947 targets, flash the board-specific runtime, connect through the serial server, deploy a hardware-specific showcase, and confirm GPIO, OLED, touch, or temperature behavior against the physical board.

Manual checks are recorded as validation notes rather than formal proofs. Their purpose is to confirm that the integrated development workflow remains usable and that the screenshots included in the thesis correspond to real UI states, not isolated mockups.

10.10 Validation Limits

The current validation suite demonstrates functional correctness for representative scenarios, but it does not constitute exhaustive verification. Full Petri net reachability graph construction is listed as future work, so the analyzer currently focuses on structural deadlocks, bottlenecks, contention, and evidence-based projections. Long-running hardware reliability is also outside the automated suite; embedded checks are smoke tests rather than multi-day endurance tests. The IDE currently has automated TypeScript validation [25], but no dedicated browser-level test runner, so visual regressions are controlled by the manual protocol. Finally, fault injection is validated with deterministic seeds and representative profiles, but the space of possible timing schedules in distributed IoT deployments remains unbounded.

These limits are acceptable for the thesis scope because the objective is to validate the architecture and demonstrate that the toolchain can execute the full Reality-in-the-Loop loop: induce a fault, observe it, record it, rewind it, and analyze its structural cause. The combination of automated tests, showcase regression checks, and manual end-to-end workflows provides evidence for that objective while clearly identifying what remains future work.

Chapter 11

End-to-End Demonstration

This chapter demonstrates Aetherium Automata as a running system. The walkthrough follows the complete operator workflow: start the infrastructure, open the IDE, load the demo project, deploy automata to a device, observe live behavior, inject a fault, replay execution history, and inspect the Petri net projection. All steps are executed against the reference Docker Compose stack that ships with the repository.

11.1 Infrastructure Setup

The reference deployment runs all backend services as Docker containers defined in `src/docker-compose.yaml`. The core development stack comprises three services:

`gateway` The Elixir/Phoenix gateway, exposed on `localhost:8080`. All IDE and device connections route through this service.

`server3` An Aetherium server instance (`SERVER_ID=svr_03`) that manages automata lifecycle and owns the execution trace store for the showcase device.

`device1` A C++17 engine process running in a container, pre-registered with `server3` and identified as `device_cpp_01`.

Starting the stack requires a single command from the `src/` directory:

```
make up # equivalent to: docker compose up -d gateway server3 device1
```

Listing 11.1: Starting the development stack.

The Makefile wraps `docker compose` for convenience and supports additional profiles for time-series monitoring (`make up-ts`) and black-box devices (`make up-blackbox`).

Once all containers report as healthy, the IDE can be launched:

```
cd src/ide
npm run dev # starts electron-vite dev server + Electron window
```

Listing 11.2: Launching the Electron IDE.

The IDE connects to the gateway at `ws://localhost:8080` using the operator token configured in the Docker environment (`GATEWAY_OPERATOR_TOKEN=dev_secret_token`). Connection status is displayed in the Gateway panel and the header status indicator.

11.2 Opening the Demo Project

The repository ships a ready-made demonstration project, `NewProject.aeth`, located in the repository root. The `.aeth` format is a JSON document that encodes all networks, devices, and their associated YAML automata paths, as described in Section 5.1.

Opening the file via *File* → *Open Project* loads all networks into the Network panel. Each network card shows its name, the number of devices it contains, and whether any of those devices are currently connected through the gateway. Figure 11.1 shows the Network panel with the demo project loaded and `device_cpp_01` online.

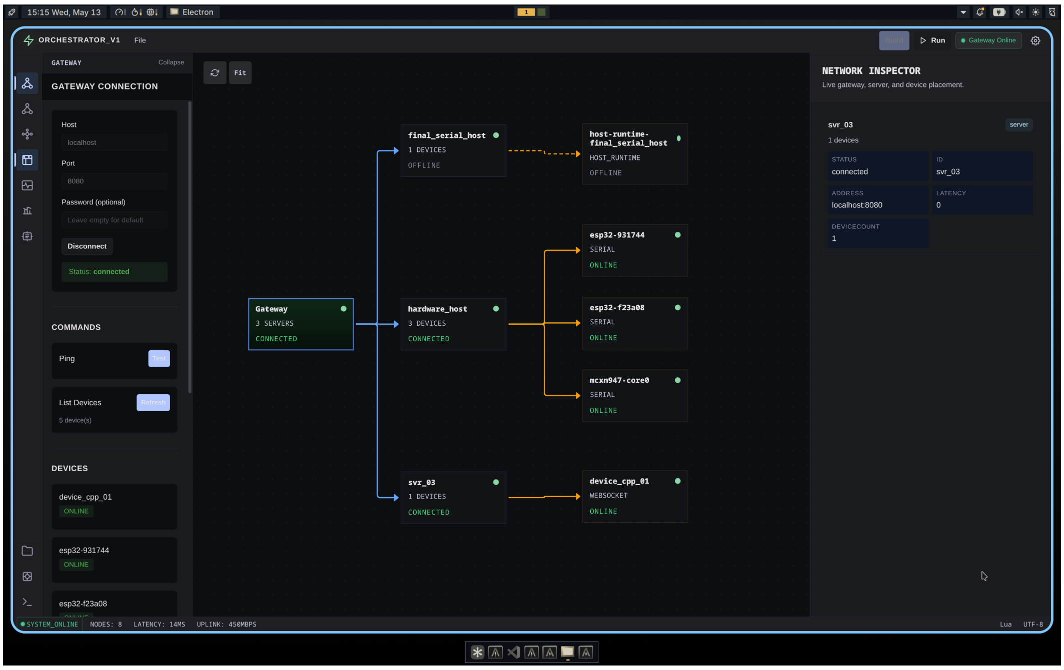

Figure 11.1: Network panel showing the demo project with one connected device.

The Explorer panel on the left lists the automata YAML files that belong to the project. Selecting a file opens it in the IDE's native code editor and inspector surface, where the YAML source and state-level Lua hooks can be inspected and edited directly. Changes are saved back to disk and can be re-deployed without restarting the stack.

11.3 Deploying Automata

Deploying an automaton to a device is an explicit operator action performed from the Runtime Monitor panel. The operator selects a target device from the device list, chooses one or more automata definitions, and clicks *Deploy*. The IDE sends a `deploy` message over the Phoenix channel, the server validates the YAML source, selects the target profile, and delivers either a YAML payload or an `aeth_ir_v1` artifact to the device engine. A deployment acknowledgment appears in the Runtime Monitor panel within a few hundred milliseconds on a local network.

Once deployed, the automaton begins executing immediately. The engine sends periodic state snapshots — containing current state, all variable values, and the last fired transition

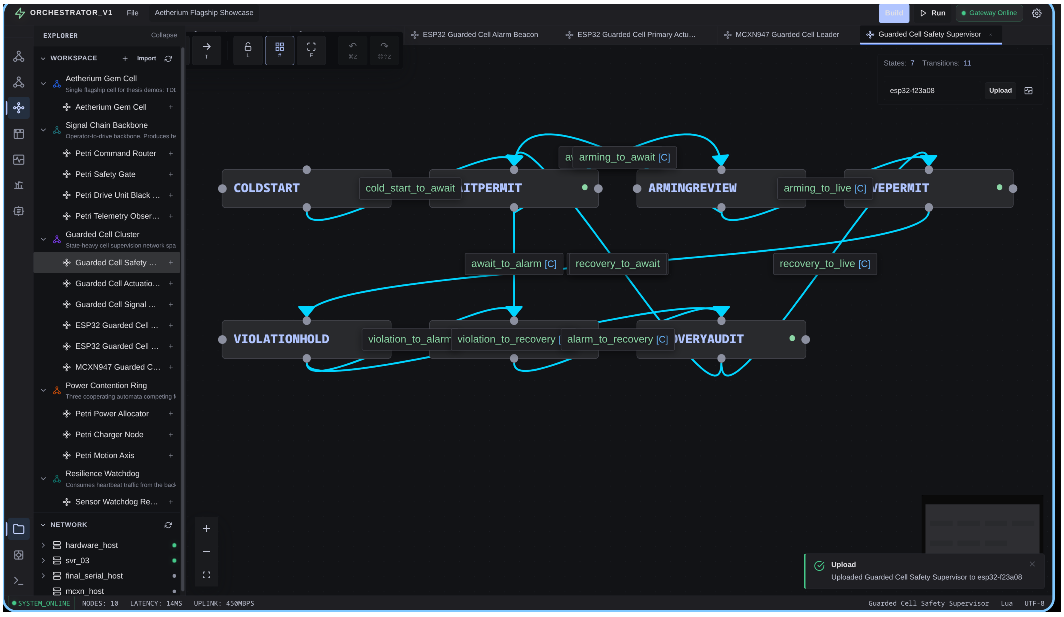

Figure 11.2: Runtime Monitor panel after deploying the sensor watchdog automaton to `device_cpp_01`.

— to the server, which forwards them to any subscribed IDE clients over the Phoenix channel.

11.4 Live Runtime Monitoring

The Runtime Monitor panel (Figure 11.3) provides a per-device, per-automaton view of live execution. For each deployed automaton, it shows:

- the **current state** name, highlighted in the state list,
- **all variable values** with their types and current readings,
- **last transition** that fired, including its type and timestamp,
- a running **tick counter** showing how many evaluation cycles have completed.

The update rate is determined by the engine's tick interval (configurable per deployment, default 100 ms) and the Phoenix channel push rate. Because the gateway maintains persistent WebSocket connections in both directions, state updates arrive in the IDE within one to two tick cycles of firing.

The monitor is read-only but supports two direct actions available via the panel toolbar: *Start* and *Stop* send lifecycle commands to the device engine, transitioning the automaton between its active and idle lifecycle states without un-deploying it.

11.5 Injecting Faults

The fault injection subsystem is accessed from the Gateway panel. The operator selects a target device and chooses from six fault profiles:

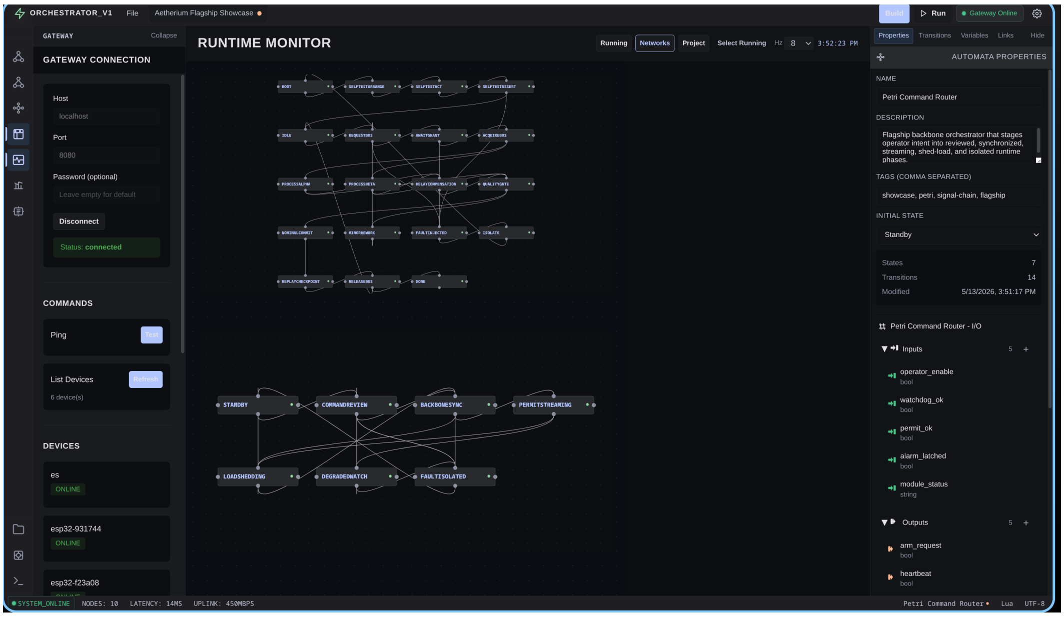

Figure 11.3: Runtime Monitor panel displaying live state and variable values for the deployed automaton.

`delay` Adds a configurable latency (milliseconds) to every outbound message from the device.

`jitter` Applies a random perturbation within a specified range to message delivery timing.

`drop` Drops a fraction of outbound messages, simulating packet loss.

`disconnect` Closes the device's WebSocket connection entirely for a specified duration, simulating a network outage.

`battery_drain` Injects a battery-level variable override to simulate low-power conditions.

`none` Clears all active fault profiles and restores normal operation.

Each profile accepts a *seed* value that initializes the random number generator used internally, making fault injection runs fully reproducible. The same seed applied to the same automaton always produces the same sequence of dropped or delayed messages, which is essential for debugging timing-dependent failures.

Figure 11.4 shows the Gateway panel with a delay fault profile active on the demo C++ device. The Runtime Monitor simultaneously shows the automaton continuing to execute despite the injected latency, demonstrating that the engine's internal timing is decoupled from transport delays.

11.6 Time-Travel Debugging

Every state transition and variable change forwarded from the device is persisted server-side in a per-deployment time series. Periodic full-state snapshots are interleaved with the event stream so that any past instant can be reconstructed without replaying the entire history from the beginning.

The time-travel workflow proceeds as follows:

Figure 11.4: Gateway panel with an active delay fault profile on the demo device.

1. The operator opens the Runtime Monitor panel. The panel shows the live execution timeline as a scrollable list of events: state transitions, variable deltas, and wall-clock timestamps.

2. The operator selects any event in the past and clicks *Rewind to Here*. The IDE sends a `rewind_deployment` WebSocket command to the gateway, carrying the target timestamp in milliseconds.

3. The server calls `TimeSeriesQuery.replay_state_at` to reconstruct the complete automaton state (active state name and all variable values) at the requested timestamp.

4. The server dispatches a `RestoreState` binary protocol message (`0x48`) to the physical device. The C++ engine pauses, applies the state snapshot atomically — setting `ctx_.currentState` and all variable values without firing any hooks — and waits in `Paused` state.

5. The operator or test sends a `Resume` command, returning the device to `Running` from the restored historical state.

Figure 11.5 shows the replay timeline for the sensor watchdog automaton. The selected event shows the automaton entering the `alarming` state and the `alarm_count` variable incrementing.

The rewind is a true device-level operation, not a visual simulation: after `RestoreState` and an explicit `Resume`, the device fires transitions consistent with the restored variable values, reproducing the execution path from the selected historical point. This makes time-travel debugging effective even for timing-sensitive faults that cannot be reproduced by simply restarting the automaton.

Figure 11.5: Execution trace replay timeline in the Runtime Monitor panel.

11.7 Petri Net Analysis

The Petri Net panel takes the deployed network as input and applies the lifting procedure described in Section 8.2. Each automaton becomes a Petri net subnet; shared output/input channel pairs become the connecting arcs between subnets. The IDE renders the resulting net as an interactive graph.

Figure 11.6 shows the Petri net projection for the *Petri Contention* showcase (Showcase 14): three automata (charger node, motion axis, power allocator) share a single power-budget channel. The Petri net makes the contention explicit: the power-budget place has three input arcs and one output arc, which the structural analysis immediately flags as a potential bottleneck.

The three structural checks available through the panel are:

Dead-state detection Flags state places with no outgoing arcs, which are structural terminal states the automaton cannot leave.

Bottleneck detection Flags channel places whose input arc count exceeds their output arc count, indicating a producer-consumer mismatch that may cause token accumulation.

Contention report Lists automata that share a resource place and reports whether their combined static demand exceeds the declared resource capacity.

The analysis results are displayed inline in the panel as a structured report beneath the net graph.

11.8 Analyzer Panel

The Analyzer panel complements the Petri net view with a higher-level projection over the live deployment. It queries the server for the current analyzer report, which the server computes from the registered automata metadata and recent transition history.

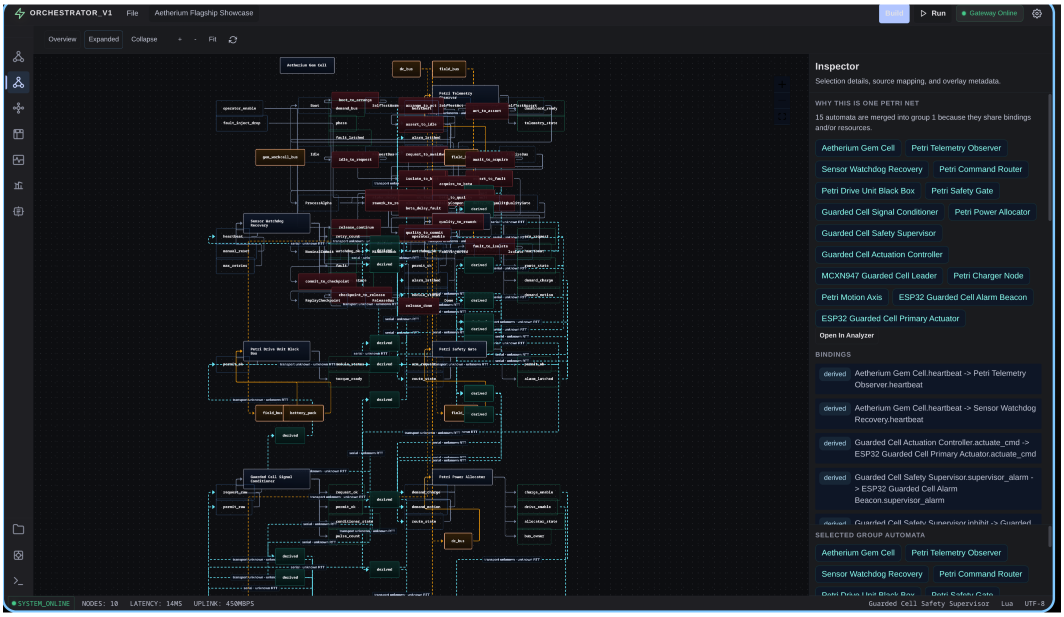

Figure 11.6: Petri Net panel showing token flow for the power-contention scenario.

The report contains three sections:

- **Device summary** — for each connected device: uptime, number of deployed automata, tick rate, and number of transitions fired since deployment.
- **Channel activity** — for each named output/input pair: message count, last value, and whether the channel is currently saturated (output producing faster than input is consuming).
- **Anomaly log** — entries flagged by the server's anomaly detector, such as a device going silent, a variable exceeding a configured threshold, or an automaton spending an unusually long time in a single state.

Figure 11.7 shows the Analyzer panel during a deployment of the Petri Signal Chain showcase (Showcase 13), where the safety gate automaton is identified as processing commands at a lower rate than the command router is emitting them.

11.9 Black-Box Devices

The black-box feature allows external systems that cannot be instrumented with the Aetherium engine to participate in the same orchestration layer. A black-box device exposes only a named signal contract; its internal implementation is opaque to the IDE.

The reference deployment includes a black-box container (`blackbox1`) that wraps a C++ engine running the Docker black-box probe automaton from showcase 12. Starting this stack variant:

```
make up-blackbox # gateway + server3 + blackbox1
make smoke-blackbox
```

Listing 11.3: Starting the black-box demonstration stack.

Figure 11.7: Analyzer panel reporting channel activity and an anomaly for the signal chain scenario.

The smoke test deploys the probe automaton, sends a sequence of activation messages, and verifies that the expected output messages arrive within the configured timeout. All interactions are visible in the Runtime Monitor panel under the `black_box_01` device entry, but the panel only shows the contract (inputs and outputs) — not the internal state of the automaton, in keeping with the black-box semantics.

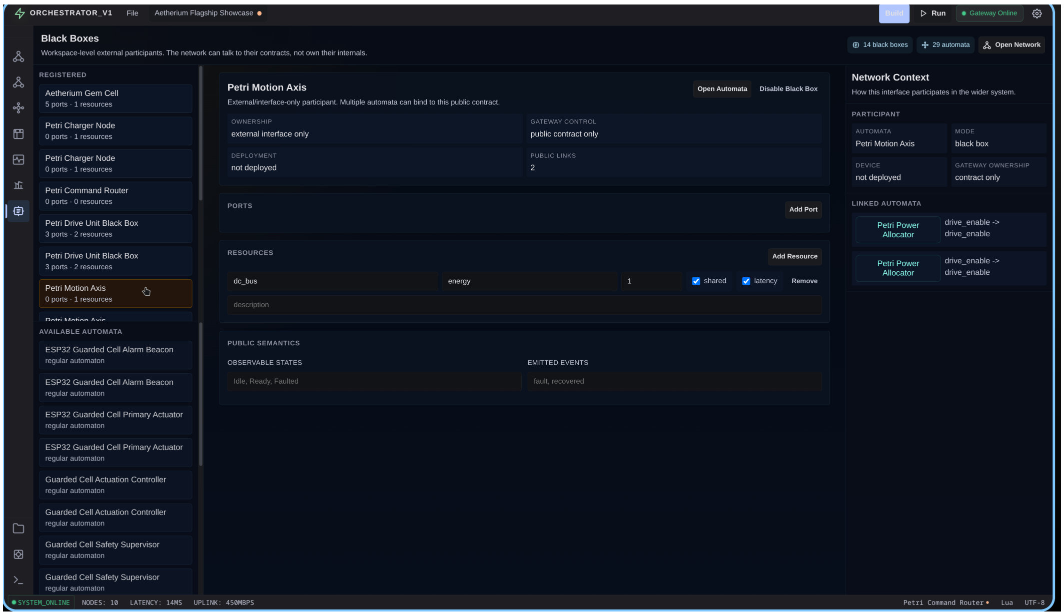

Figure 11.8: Runtime Monitor panel showing the black-box device contract for `black_box_01`.

11.10 Demonstration Summary

The end-to-end walkthrough demonstrates all primary platform capabilities on a single machine without any physical hardware. Table 11.1 maps each demonstrated capability to the platform component responsible for it.

Table 11.1: Demonstration coverage summary.

Capability	Platform component	Demo section
Stack startup	Docker Compose / Makefile	§11.1
Project file open	IDE file handler	§11.2
Automaton deployment	Server + engine deploy API	§11.3
Live state monitoring	Runtime Monitor panel	§11.4
Fault injection	Gateway fault service	§11.5
Time-travel debugging	Execution trace + replay	§11.6
Petri net analysis	Petri Net panel + lifting	§11.7
Anomaly reporting	Analyzer panel	§11.8
Black-box contract	Black-box device type	§11.9

The hardware-in-the-loop variants (ESP32, MCXN947) follow the same IDE workflow; the only difference is that `make up-gateway` replaces the full Docker stack, and the server runs on the host machine with a serial transport connector. This confirms that the platform's layered architecture — described in Chapter 4 — successfully isolates the operator-facing workflow from the physical transport.

Chapter 12

Conclusion

12.1 Summary

This thesis presented Aetherium Automata, a platform for the visual design, deployment, and monitoring of distributed IoT control systems modeled as Extended Finite State Machines. The system was designed around two interrelated principles introduced in Section 1.1.

Reality-in-the-Loop holds that production code and testing infrastructure must share the same execution substrate. Rather than maintaining a separate test harness, Aetherium lets developers run the actual automaton on actual (or emulated) hardware while the fault injection layer perturbs the communication boundary at configurable rates. Probabilistic transitions make rare real-world events reliably inducible; black-box contracts define the formal interface under test; and time-travel debugging closes the loop by allowing the developer to rewind to the exact state sequence that preceded any observed anomaly. Events that would otherwise require weeks of field observation can be forced, observed, and diagnosed in a single development session.

Deployment-Aware Development holds that the developer should never be blind to the network their automata run on. Because Aetherium owns the communication layer end-to-end, every deployed device is accompanied by live telemetry — RTT, connection type, wake-up timer intervals, and battery level, where the hardware exposes it — available in the same IDE panel as the state machine. Developers can make quantified, deliberate trade-offs instead of guessing at network conditions. The Petri net analysis layer extends this to the structural level: lifting all automata in the network to a single Petri net makes bottleneck channels, dead states, and resource contention structurally visible, giving the developer a vantage point that is inaccessible when inspecting each device in isolation.

The concrete contributions that realize these principles are:

1. A formal EFSM model [14] supporting five transition types (classic, timed, event, probabilistic, immediate), typed variables with reactive change detection, and Lua-scripted [17] guards and actions. The probabilistic type is the engine of RitL.

2. A YAML [43] domain-specific language for authoring automata in both inline and folder layouts, with a parser that validates structure and identifier correctness.

3. A portable C++17 runtime engine that executes the model on desktop Linux and embedded platforms (ESP32 [10], MCXN947 [30]) via three abstract platform interfaces (`IClock`, `IRandomSource`, `IScriptEngine`).

4. A binary wire protocol inspired by MQTT's [31] lightweight publish/subscribe model but reimplemented to support server-side sequence numbering, execution trace recording, and target-aware deployment of both YAML and `aeth_ir_v1` payloads.

5. An Electron [35] IDE providing live state monitoring, variable inspection, fault injection, time-travel debugging, and Petri net analysis — the single surface at which both RitL and DAD are exercised.

6. A Petri net analysis layer [27] that lifts multi-automata networks to a structural plane where bottleneck channels, dead states, and resource contention are structurally identified — the highest expression of Deployment-Aware Development.

7. A showcase set of 39 YAML automata covering real-world IoT scenarios, including resilience, energy management, probabilistic routing, multi-device coordination, and the dedicated Aetherium Gem Cell project.

8. A multi-layer validation suite combining unit tests, component tests, end-to-end deployment checks, curated showcase validation, IDE TypeScript validation, manual IDE workflows, and hardware smoke tests.

12.2 Evaluation Against Goals

The goals stated in Section 1.1 and Section 1.2 can be evaluated directly against the implemented system.

The first goal was to design an EFSM model expressive enough for real IoT control logic. This goal was met by the combination of typed variables, Lua guards and actions, five transition types, and state-level hooks. The showcase catalog demonstrates this expressiveness across watchdog recovery, probabilistic quality routing, power allocation, signal chains, bidirectional feedback, and the Gem Cell.

The second goal was portability. The runtime core is written in C++17 and separated from platform-specific services through clock, random-source, and script-engine abstractions. Desktop execution is used for Docker and local testing; embedded profiles cover ESP32 and MCXN947. The deployment compiler tests confirm that richer targets can accept richer automata while constrained profiles reject unsupported constructs deterministically.

The third goal was a practical authoring format. The YAML DSL satisfies this by keeping automata definitions readable in plain text while still supporting inline and folderized layouts. The DSL is also compatible with the IDE: developers can edit YAML directly, but the same file can be visualized as states, transitions, variables, and network wiring.

The fourth goal was live deployment and monitoring. This was met by the Phoenix gateway, server-side device manager, binary device protocol, and IDE runtime monitor. The operator can deploy an automaton, observe the current state and variables, modify inputs, and inspect transition history without leaving the tool.

The fifth goal was Reality-in-the-Loop testing. The implementation supports seeded fault profiles, black-box contracts, trace recording, state reconstruction, and device-level restore. The validation chapter shows that this loop is tested through unit, integration, end-to-end, and manual workflows.

The sixth goal was Petri net analysis. The system lifts automata into Petri net substructures, composes shared channels and resources, and reports dead states, bottleneck channels, and resource contention from static YAML definitions. This does not constitute a complete formal verification engine — full reachability analysis and P-invariant computation are future work — but it does provide useful structural visibility inside the development workflow.

12.3 Results

The following concrete outcomes were produced and verified by the implementation and validation described in the preceding chapters.

EFSM model and DSL. Five transition types (classic, timed, event, probabilistic, immediate), seven variable types, and three code hooks per state (`on_enter`, `body`, `on_exit`) are fully implemented. The YAML parser enforces naming constraints and referential integrity at load time. Priority-based resolution with deterministic weighted selection is verified across the showcase catalog.

Portable runtime. The C++17 engine compiles for desktop Linux, ESP32 (Xtensa LX6, Arduino/FreeRTOS), and NXP FRDM-MCXN947 [30] (ARM Cortex-M33, bare-metal) from a single source tree via three abstract platform interfaces. Hardware peripheral APIs (gpio, pwm, adc, dac, i2c) and five component types (board_led, i2c_scanner, board_temp, touch_pad, ssd1306_text) are exposed to Lua scripts; platform capability is enforced at runtime so that a misconfigured peripheral fails fast with a clear diagnostic.

Fault injection. Four fault categories (delay, jitter, message drop, disconnect) operate at the communication boundary with configurable probability parameters and a deterministic seed for reproducible fault scenarios. A seeded fault run applied to the same automaton always produces the same perturbation sequence.

Time-travel debugging. The full rewind chain is implemented: every state transition and variable change is stored in the server-side time-series; `replay_state_at` reconstructs the exact automaton state at a past timestamp; a `RestoreState` binary message (`0x48`) atomically resets the device to that state without firing hooks; execution can then be resumed explicitly from the restored point.

Petri net analysis. Dead-state detection, structural bottleneck identification, and pairwise resource-contention reporting are implemented as graph-structural passes over the composed Petri net derived from YAML definitions. No execution trace is required; the analysis runs before any device is connected. Full reachability analysis and P-invariant computation are left as future work.

Showcase catalog. 39 YAML automata across 15 categories cover timed, event, and probabilistic transitions; watchdog recovery; pipeline dispatch; multi-device supervision; ESP32 hardware peripherals; NXP FRDM-MCXN947 peripherals; bidirectional signaling; black-box contracts; Petri signal chains; power contention; and the Aetherium Gem Cell flagship.

Validation. The multi-layer validation suite spans C++ build and CLI/YAML parser checks, Elixir ExUnit tests for the gateway and server (protocol encoding, device registry, replay, analyzer), the `mix aetherium.e2e` end-to-end deployment task, curated showcase catalog validation, IDE TypeScript validation [25], Docker-based stack smoke checks (core, black-box, time-series), and manual IDE and hardware workflows.

All nine requirements stated in Section 3.4 are addressed: R1 (EFSM expressiveness with five transition types), R2 (Lua scripting), R3 (portability to desktop, ESP32, and MCXN947), R4 (YAML DSL), R5 (real-time monitoring with deployment metadata), R6 (fault injection), R7 (time-travel debugging with physical device restore), R8 (Petri net structural analysis), and R9 (black-box contracts).

12.4 Limitations

Several limitations remain. The Petri analysis is intentionally bounded and structural. Complete reachability graph construction for large networks is still future work because state-space explosion becomes significant once many automata and data-dependent guards are involved. The current analyzer is therefore best understood as a practical design-time warning system rather than a proof of all possible executions.

The IDE is functional but still a research prototype. It demonstrates visual editing, live monitoring, fault injection, replay, and Petri views, but it does not yet provide all productivity features expected from a mature modeling environment: complete undo/redo history, multi-select editing, advanced layout tools, and collaborative project management.

Hardware validation is also limited to representative smoke workflows. The runtime architecture supports embedded targets, and board-specific examples exist, but the thesis does not claim long-term reliability data from multi-day hardware deployments. That kind of endurance validation would require a separate experimental setup with controlled power, network, and environmental conditions. During live ESP32 and FRDM-MCXN947 demonstrations, the USB serial/debug probe itself can also be a transient failure point: a port may remain visible while the firmware no longer announces itself, or a deploy may time out waiting for a serial chunk acknowledgement until the board is reset, replugged, or reflashed with a conservative deploy chunk size. This is documented as an operational recovery procedure rather than hidden as a functional limitation of the EFSM model.

Finally, the current DSL uses Lua for guard and action logic. This is pragmatic and expressive, but it also means that not all behavior is statically analyzable. Future versions could introduce a restricted expression subset for safety-critical transitions while keeping Lua available for richer targets and non-critical behavior.

12.5 Practical Impact

The practical value of Aetherium is that it shortens the distance between modeling, deployment, and diagnosis. In many IoT projects, these activities are separated: a state diagram may exist in documentation, device code is written separately, telemetry is collected by another system, and debugging depends on logs. Aetherium keeps these activities connected to the same EFSM definition. The state graph is not only documentation; it is the deployed behavior, the runtime monitoring surface, and the source model for analysis.

This is particularly useful for small and medium-sized IoT teams. Such teams often do not have the capacity to maintain separate formal models, firmware implementations, cloud dashboards, and test harnesses. A single toolchain that can deploy a model, perturb it with faults, capture its execution trace, and replay it from a previous state reduces that maintenance burden. It also improves communication: a developer, tester, and supervisor can discuss the same visible state machine instead of translating between code, logs, and diagrams.

The platform also makes failure handling more explicit. Watchdogs, retry loops, degraded modes, black-box interfaces, and recovery transitions are represented as states and edges rather than hidden inside callbacks. This encourages developers to model negative paths early. Fault injection then turns those negative paths into executable scenarios. A failed sensor, delayed message, disconnected device, or unavailable resource becomes a planned test condition rather than a surprise in production.

Another practical contribution is observability at the correct abstraction level. Raw logs are too low-level for many control problems, while dashboard metrics are often too high-level to explain a state-machine failure. Aetherium records the middle layer: transitions, variables, commands, snapshots, and deployment metadata. This information is precise enough for debugging and compact enough for replay.

Finally, the Petri net lift provides a bridge between engineering practice and formal analysis. Developers do not need to write Petri nets manually. They model devices as automata, and the tool derives a structural view that exposes overloaded channels, shared resources, and dead states. This lowers the barrier to applying structural formal analysis in everyday IoT development.

12.6 Future Work

The following directions are identified for future development:

- Full Petri net reachability analysis: BFS/DFS enumeration of all reachable markings, dynamic deadlock detection (markings with no enabled transitions), k-boundedness verification over all reachable markings, and P-invariant computation via incidence-matrix analysis. The current analyzer performs structural graph checks only; state-space enumeration for data-dependent guards at scale requires dedicated tooling and is the natural next step for the analysis layer.

- Bytecode compilation of Lua scripts into a compact `EngineBytecodeProgram` format for deployment to the most constrained targets.

- Hierarchical and abstract automata — template (parameterized) automata instantiated at compose time, with Petri net lifting extended to hierarchical networks, moving the system toward a full DEVS-style coupled-model composition.

- Third-party device interoperability via black-box adapters: the black-box contract architecture (Section 5.6) makes it straightforward to wrap external subsystems — ROS2 nodes, Zigbee gateways, WiFi relays — with a thin protocol adapter that exposes only the signal contract, without modifying the core engine or gateway.

- Raspberry Pi Pico (RP2040) target: the abstract platform interfaces (`IClock`, `IRandomSource`, `IScriptEngine`) are designed to accommodate a Pico port. GPIO, PWM, and ADC mappings are straightforward; the main outstanding work is the serial transport link and validation on hardware.

- Extended IDE canvas: undo/redo history, multi-select, and copy-paste of states and transitions.

- Expanded InfluxDB and Grafana support for long-running time-series metrics dashboards, building on the optional time-series profile already used by the validation stack [18, 13].

Bibliography

[1] Amazon Web Services, Inc.. *FreeRTOS: Real-Time Operating System for Microcontrollers* online. 2024. Available at: https://www.freertos.org/. [cit. 2025-05-12].

[2] Arduino LLC. *Arduino: Open-Source Electronics Platform* online. 2024. Available at: https://www.arduino.cc/. [cit. 2025-05-12].

[3] Armstrong, J. *Programming Erlang: Software for a Concurrent World*. Raleigh, NC: Pragmatic Bookshelf, 2007. ISBN 978-1-934356-00-5.

[4] Beilharz, J.; Wiesner, P.; Boockmeyer, A.; Friedenberger, D.; Brokhausen, F. et al. Continuously Testing Distributed IoT Systems: An Overview of the State of the Art. In: *Service-Oriented Computing – ICSOC 2021 Workshops*. Cham: Springer, 2022, p. 375–387.

[5] Biga, J. P. M. *RapidYAML: Rapid YAML — a library to parse and emit YAML, and do it fast* online. 2024. Available at: https://github.com/biojppm/rapidyaml. [cit. 2025-05-11]. MIT License.

[6] Docker, Inc.. *Docker: Accelerated Container Application Development* online. 2024. Available at: https://www.docker.com/. [cit. 2025-05-12].

[7] Eclipse Foundation. *Eclipse 4diac: Open Source Infrastructure for Distributed Industrial Control Based on IEC 61499* online. 2024. Available at: https://eclipse.dev/4diac/. [cit. 2025-05-11].

[8] Engblom, J. A Review of Reverse Debugging. In: *2012 System, Software, SoC and Silicon Debug Conference (S4D)*. IEEE, 2012, p. 1–6.

[9] Erlang/OTP Documentation. *Processes — Erlang System Documentation* online. 2026. Available at: https://www.erlang.org/doc/system/eff_guide_processes.html. [cit. 2026-05-13].

[10] Espressif Systems. *ESP32 Technical Reference Manual* online. 2024. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [cit. 2025-05-12].

[11] Fette, I. and Melnikov, A. *The WebSocket Protocol (RFC 6455)* online. 2011. Available at: https://datatracker.ietf.org/doc/html/rfc6455. [cit. 2025-05-12]. IETF Proposed Standard.

[12] Geels, D.; Altekar, G.; Shenker, S. and Stoica, I. Replay Debugging for Distributed Applications. In: *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. Boston, MA: USENIX Association, 2006, p. 289–300. Available at: https://www.usenix.org/conference/2006-usenix-annual-technical-conference/replay-debugging-distributed-applications. Best Paper Award.

[13] Grafana Labs. *Grafana Documentation: Dashboards* online. 2026. Available at: https://grafana.com/docs/grafana/latest/visualizations/dashboards/. [cit. 2026-05-13].

[14] Harel, D. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*. North-Holland, 1987, vol. 8, no. 3, p. 231–274. ISSN 0167-6423.

[15] Hopcroft, J. E.; Motwani, R. and Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Upper Saddle River, NJ: Pearson Education, 2006. ISBN 978-0-321-45536-9.

[16] Ierusalimschy, R.; de Figueiredo, L. H. and Celes, W. *Lua 5.4 Reference Manual* online. Lua.org, PUC-Rio, 2020. Available at: https://www.lua.org/manual/5.4/. [cit. 2025-05-11].

[17] Ierusalimschy, R.; de Figueiredo, L. H. and Celes Filho, W. Lua—An Extensible Extension Language. *Software: Practice and Experience*. Wiley, 1996, vol. 26, no. 6, p. 635–652. ISSN 1097-024X. Available at: https://doi.org/10.1002/(SICI)1097-024X(199606)26:6<635::AID-SPE26>3.0.CO;2-P.

[18] InfluxData Inc.. *Time Series Database Explained* online. 2026. Available at: https://www.influxdata.com/time-series-database/. [cit. 2026-05-13].

[19] International Electrotechnical Commission. *IEC 61499-1: Function Blocks, Part 1: Architecture, Edition 2* online. 2012. Available at: https://www.iec.ch/homepage/prod/other.htm. [cit. 2025-05-12]. IEC 61499-1 Ed.2.

[20] International Electrotechnical Commission. *IEC 61131-3: Programmable Controllers, Part 3: Programming Languages, Edition 3* online. 2013. Available at: https://www.iec.ch/homepage/prod/other.htm. [cit. 2025-05-12]. IEC 61131-3 Ed.3.

[21] Kitware, Inc.. *CMake: The Cross-Platform Open-Source Build System* online. 2024. Available at: https://cmake.org/. [cit. 2025-05-12].

[22] LeBlanc, T. J. and Mellor Crummey, J. M. Debugging Parallel Programs with Instant Replay. *IEEE Transactions on Computers*. IEEE, 1987, vol. 36, no. 4, p. 471–482. ISSN 0018-9340.

[23] Machine Zone, Inc.. *IXWebSocket: WebSocket and HTTP Client and Server Library* online. 2024. Available at: https://github.com/machinezone/IXWebSocket. [cit. 2025-05-11]. BSD 3-Clause License.

[24] Meta Open Source. *React: The Library for Web and Native User Interfaces* online. 2024. Available at: https://react.dev/. [cit. 2025-05-11].

[25] MICROSOFT. *TypeScript Handbook: The Basics* online. 2026. Available at: https://www.typescriptlang.org/docs/handbook/2/basic-types.html. [cit. 2026-05-13].

[26] MORRISON, J. P. *Flow-Based Programming: A New Approach to Application Development*. New York, NY: Van Nostrand Reinhold, 1994. ISBN 978-0-442-01771-2.

[27] MURATA, T. Petri Nets: Properties, Analysis and Applications. *Proceedings of the IEEE*. IEEE, 1989, vol. 77, no. 4, p. 541–580. ISSN 0018-9219.

[28] NOAMAN, M.; KHAN, M. S.; ABRAR, M. F.; ALI, S.; ALVI, A. et al. Challenges in Integration of Heterogeneous Internet of Things. *Scientific Programming*. Hindawi, 2022, vol. 2022, p. 1–17.

[29] NODE-RED PROJECT. *Node-RED Library* online. 2026. Available at: https://flows.nodered.org/. [cit. 2026-05-13].

[30] NXP SEMICONDUCTORS. *FRDM Development Board for MCX N94/N54 MCUs* online. 2026. Available at: https://www.nxp.com/design/design-center/development-boards-and-designs/FRDM-MCXN947. [cit. 2026-05-13]. FRDM-MCXN947 board product page.

[31] OASIS. *MQTT Version 5.0* online. 2019. Available at: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html. [cit. 2025-05-12]. OASIS Standard.

[32] O'CALLAHAN, R.; JONES, C.; FROYD, N.; HUEY, K.; NOLL, A. et al. *Lightweight User-Space Record and Replay* online. 2016. Available at: https://arxiv.org/abs/1610.02144. [cit. 2026-05-13]. ArXiv:1610.02144.

[33] O'LEARY, N. *Moving to the JS Foundation* online. 2016. Available at: https://nodered.org/blog/2016/10/17/js-foundation. [cit. 2026-05-13]. Node-RED project blog.

[34] O'LEARY, N. and CONWAY-JONES, D. *Node-RED: Low-Code Programming for Event-Driven Applications* online. 2013. Available at: https://nodered.org/. [cit. 2025-05-11]. Open-sourced September 2013; now part of the OpenJS Foundation.

[35] OPENJS FOUNDATION. *Electron: Build Cross-Platform Desktop Apps with JavaScript, HTML, and CSS* online. 2024. Available at: https://www.electronjs.org/. [cit. 2025-05-11].

[36] PETRI, C. A. *Kommunikation mit Automaten*. Darmstadt, Germany, 1962. Dissertation. Technische Hochschule Darmstadt.

[37] PHOENIX FRAMEWORK CONTRIBUTORS. *Phoenix Framework — Peace of Mind from Prototype to Production* online. 2024. Available at: https://www.phoenixframework.org/. [cit. 2025-05-11].

[38] POIMANDRES (PMNDRS). *Zustand: Bear Necessities for State Management in React* online. 2024. Available at: https://github.com/pmndrs/zustand. [cit. 2025-05-11]. MIT License.

[39] Strasser, T.; Rooker, M. N.; Ebenhofer, G.; Zoitl, A.; Sunder, C. et al. Framework for Distributed Industrial Automation and Control (4DIAC). In: *2008 6th IEEE International Conference on Industrial Informatics (INDIN)*. IEEE, 2008, p. 283–288.

[40] ThePhD. *Sol2: C++ ↔ Lua API Wrapper* online. 2023. Available at: https://github.com/ThePhD/sol2. [cit. 2025-05-11]. MIT License.

[41] Tu, Y.; Li, D. and Li, S. Formal Syntax and Semantics of Basic Function Blocks in IEC 61499. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, 2012, vol. 226, no. 4, p. 542–555.

[42] Valim, J. et al. *Elixir Programming Language* online. 2024. Available at: https://elixir-lang.org/. [cit. 2025-05-11].

[43] YAML Language Development Team. *YAML Ain't Markup Language (YAML) Revision 1.2.2* online. 2021. Available at: https://yaml.org/spec/1.2.2/. [cit. 2025-05-11].

[44] Zoitl, A. and Strasser, T. *Distributed Control Applications: Guidelines, Design Patterns, and Application Examples with the IEC 61499*. Boca Raton, FL: CRC Press / Taylor & Francis, 2017. ISBN 978-1-482-25905-6.

Appendix A

Contents of the Included Storage Media

The included storage medium contains the following items:

`src/` Full source code of the Aetherium Automata platform:

- `src/engine/` — C++17 runtime engine and platform targets
- `src/ide/` — Electron IDE (TypeScript/React)
- `src/gateway/` — Elixir/Phoenix gateway
- `src/server/` — Elixir server

`example/` Showcase automata catalog:

- `example/automata/showcase/` — 39 YAML automata definitions
- `example/ide_demo_projects/` — IDE demo project files (`.aeth`)

`thesis/` LaTeX source of this technical report and compiled PDF.

Appendix B

YAML DSL Schema Reference

The complete field reference for the YAML domain-specific language is given below. Fields marked **(R)** are required; fields marked **(O)** are optional.

Table B.1: Top-level YAML fields.

Field	Req.	Description
`version`	R	Schema version (currently `0.0.1`)
`config`	R	Automaton metadata block
`variables`	O	List of variable definitions
`automata`	R	EFSM body

Table B.2: `config` block fields.

Field	Req.	Description
`name`	R	Human-readable automaton name
`type`	R	`inline` or `folder`
`location`	R if folder	Path to the folder containing code files
`language`	O	Scripting language; default `lua`
`description`	O	Free-text description
`author`	O	Author name
`tags`	O	List of string tags for discovery

Table B.3: Transition type-specific fields.

Type	Required fields	Optional fields
`classic`	`condition`	`priority`, `weight`
`timed`	`timed.mode`; timing field depends on mode (`after`, `every`, ...)	`timed.jitter`, `priority`
`event`	`event.triggers`	`event.require_all`, `priority`
`probabilistic`	`weight`	`priority`
`immediate`	—	`priority`

Appendix C

Supported Platform Targets

Table C.1: Engine platform targets and their capability profiles.

Target	Architecture	Script engine	Transport	Build
Desktop Linux/macOS	x86-64 / ARM64	Lua 5.4 via Sol2	WebSocket	CMake
ESP32	Xtensa LX6	Lightweight or stub	Serial / MQTT	Arduino
Raspberry Pi Pico	ARM Cortex-M0+	No-op stub	Serial (UART)	CMake
MCXN947	ARM Cortex-M7	Lightweight	Serial (UART)	CMake

Appendix D

Required Demonstration Figures

The current thesis includes placeholder IDE figures so the report can be compiled before final screenshots are captured. The final visual pass should replace these placeholders with screenshots taken from the running Aetherium Gem Cell and flagship desktop project.

Table D.1: Screenshot checklist for the final thesis figures.

Figure	Required capture
Network panel	Open the backend capabilities tour; capture the project tree with *Aetherium Gem Cell* expanded above the signal-chain and power-contention networks.
Runtime deployment	Deploy the Gem Cell automaton to the demo C++ device; capture the deployment acknowledgement and selected automaton.
Runtime monitor	Run the Gem Cell with the operator enabled; capture the state list around request, acquire, and replay phases, including phase, TDD, demand, and replay variables.
Fault injection	Enable a drop or delay fault against the Gem Cell boundary; capture the Gateway fault profile and the monitor showing fault or compensation behavior.
Time travel	Rewind to the replay checkpoint marker; capture the trace timeline with the selected event and reconstructed state.
Petri net	Open the Petri Net panel for the Gem Cell plus Power Contention Ring; capture the shared bus/resource place and contention report.
Analyzer	Capture a report where the analyzer shows the Gem Cell blackbox ports, emitted events, and shared-resource metadata.
Hardware variant	Optional: capture ESP32 or MCXN947 connected through serial to show the same IDE workflow with a physical target.